# DFS Templates

DFS appears in five forms. The pattern determines naming, return type, and where the answer is built.

## When to Use — Signal → Pattern

| Problem signal phrase | Pattern / template |
|---|---|
| "height / depth / sum OF the subtree", combine results upward | #1 Process children first, combine at node (post-order) |
| "root-to-leaf path", "number built along the path" | #2 Process node first, pass state down (pre-order) |
| "longest/best path that may bend through a node" (spans both arms) | #3 Tree global answer |
| "count islands / largest region / flood fill" on a grid | #4 Grid DFS |
| "can finish all", "detect cycle", "valid course order" (directed) | #5 Graph DFS 3-color |
| "all subsets / permutations / combinations", "find every way" | #6 Backtracking |
| "valid BST", "kth smallest", "closest value", "successor" | BST templates (pass bounds / inorder / binary search) |

## Quick Reference Table

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 104 | Maximum Depth of Binary Tree | Maximum number of nodes along the longest root-to-leaf path. | my depth is just one more than my deeper child's depth. | O(n) | O(h) | Standard |
| 543 | Diameter of Binary Tree | Length of the longest path between any two nodes (path does not need to pass through root). | the longest path bends at some node using both arms, but only one arm can extend to its parent. | O(n) | O(h) | Standard |
| 124 | Binary Tree Maximum Path Sum | Maximum sum of any path between any two nodes. Values can be negative. | a negative arm is worse than taking nothing, so clamp it to 0 before adding. | O(n) | O(h) | Standard |
| 110 | Balanced Binary Tree | Is every node's left and right subtree within 1 height of each other? | fold "is it balanced" into the height return — a poisoned -1 bubbles up and stops the work. | O(n) | O(h) | Standard |
| 226 | Invert Binary Tree | Mirror a binary tree — swap left and right subtrees at every node. | invert both children, then swap the pointers at this node. | O(n) | O(h) | Standard |
| 112 | Path Sum | Does any root-to-leaf path sum to `targetSum`? | subtract each node from the target on the way down; a leaf with remaining 0 is a hit. | O(n) | O(h) | Standard |
| 113 | Path Sum II | Return all root-to-leaf paths that sum to `targetSum`. | the same `path` list is reused for every branch, so undo your add as you unwind. | O(n) | O(n) for output paths | Standard |
| 257 | Binary Tree Paths | Return all root-to-leaf paths as strings (`"1->2->5"`). | append to a shared StringBuilder, then truncate back to your entry length to undo. | O(n) | O(n) for output paths | Standard |
| 129 | Sum Root to Leaf Numbers | Each root-to-leaf path represents a number (e.g., 1→2→3 = 123). Return the total sum. | build the number digit by digit on the way down (`current*10 + val`); add it up at each leaf. | O(n) | O(h) | Standard |
| 200 | Number of Islands | Count the number of islands (connected groups of `'1'`) in a binary grid. | each unvisited land cell starts a new island; flood-fill sinks the whole island so it's counted once. | O(m·n) | O(m·n) | Standard |
| 695 | Max Area of Island | Return the area of the largest island. | an island's area is 1 (this cell) plus the areas its four neighbors flood back. | O(m·n) | O(m·n) | Standard |
| 207 | Course Schedule | Can you finish all courses given prerequisites? Return false if there is a cycle. | re-entering a node still on the current DFS stack means you looped back to a prerequisite → cycle. | O(V+E) | O(V+E) | Standard |
| 210 | Course Schedule II | Return a valid order to take all courses. Return `[]` if a cycle exists. | a node finishes only after all it depends on do, so post-order reversed is a valid prerequisite order. | O(V+E) | O(V+E) | Standard |
| 46 | Permutations | All permutations of distinct integers. | order matters, so every position considers all unused elements (no `start` index). | O(n·n!) | O(n) | Standard |
| 78 | Subsets | All subsets (power set) of distinct integers. | every prefix on the way down is itself a valid subset, so record at every node. | O(n·2ⁿ) | O(n) | Standard |
| 39 | Combination Sum | All combinations summing to target. Same element may be used unlimited times. No duplicate combos. | passing `i` (not `i+1`) lets you pick the same element again; `start` still forbids going backward, so no reordered dups. | O(n · 2ⁿ) | O(target/min) | Standard |
| 40 | Combination Sum II | Each element used at most once. Input may have duplicates. No duplicate combos in output. | after sorting, equal values sit together; skipping a repeat at the *same level* prevents producing the same combo twice. | O(n · 2ⁿ) | O(target/min) | Standard |
| 79 | Word Search | Given a board of characters, return true if the word exists as a connected path (no cell reused). | the grid itself is the state — temporarily blank a cell so the same path can't reuse it, then restore on the way back. | O(m·n·4^L) where L = word length | O(L) | Standard |
| 131 | Palindrome Partitioning | Partition string `s` such that every substring is a palindrome. Return all such partitions. | try every prefix as the first cut; only recurse on the rest when that prefix is a palindrome. | O(n · 2ⁿ) | O(n) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 98 | Validate Binary Search Tree | Determine if a binary tree is a valid BST. Each node's value must be strictly greater than all values in its left subtree and strictly less than all values in its right subtree. | | O(n) | O(h) | Pass min/max bounds down — each call tightens the valid range. Use `long` to handle `Integer.MIN_VALUE` and `Integer.MAX_VALUE` edge cases. |
| 230 | Kth Smallest Element in a BST | Find the kth smallest element in a BST (1-indexed). | | O(k) average, O(n) worst | O(h) | BST inorder traversal visits nodes in ascending order. Count nodes visited; when count reaches k, record the answer and stop early. |
| 270 | Closest Binary Search Tree Value | Given a BST and a `target` (double), return the value in the BST closest to target. | | O(h) | O(1) | Use BST structure to navigate in O(h). At each node, compare distance to closest seen so far, then binary-search left or right. |
| 285 | Inorder Successor in BST | Given a node `p` in a BST, return its inorder successor (the smallest node with value greater than `p.val`). Return null if no such node exists. | | O(h) | O(1) | Navigate the BST. When `root.val > p.val`, this root is a candidate — save it and go left (to find a smaller valid candidate). When `root.val <= p.val`, go right (must find something larger). |
| 687 | Longest Univalue Path | Return the length of the longest path in a binary tree where every node along the path has the same value. The path does not need to pass through the root. | | O(n) | O(h) | Post-order DFS. At each node, extend the left or right path only if the child's value matches. The path through the current node = leftPath + rightPath. Return the max single-direction extension upward. |
| 37 | Sudoku Solver | Fill a partially filled 9×9 Sudoku so every row, column, and 3×3 box contains digits 1-9. | | O(9^m) m=empty cells | O(1) | backtracking that tries digits 1-9 in each empty cell, validating against row, column, and box constraints before recursing. |
| 47 | Permutations II | Return all unique permutations of an array that may contain duplicates. | | O(n·n!) | O(n) | sort first; use a `used[]` array; skip a value if the previous equal value at the same tree level has not been used (prevents duplicate permutations). |
| 51 | N-Queens | Place n queens on an n×n board so no two attack each other; return all distinct solutions. | | O(n!) | O(n) | backtrack row by row; track occupied columns and both diagonals. Cells on the same `\` diagonal share `r−c`; cells on the same `/` diagonal share `r+c`. |
| 60 | Permutation Sequence | Return the kth permutation (1-indexed) of the sequence 1..n. | | O(n²) | O(n) | no backtracking — use the factorial number system. Each position's digit is determined directly by `(k−1) / (remaining−1)!`. |
| 90 | Subsets II | Return all possible unique subsets of an array that may contain duplicates. | | O(n·2ⁿ) | O(n) | sort first; within a recursion level, skip a value equal to its predecessor to avoid duplicate subsets. |
| 216 | Combination Sum III | Find all combinations of k numbers from 1-9 (each used at most once) that sum to n. | | O(C(9,k)) | O(k) | standard combination backtracking with start index `i+1` (no reuse), but with two stopping constraints — count and remaining sum. |
| 549 | Binary Tree Longest Consecutive Sequence II | Find the length of the longest consecutive path in a binary tree. The path can be increasing or decreasing and may go through a node (child-parent-child). | | O(n) | O(h) | post-order DFS returns a pair `{increasing, decreasing}` for each node. A path through the node combines an increasing run from one child with a decreasing run from the other. |
| 17 | Letter Combinations of a Phone Number | Given a string of digits 2-9, return all letter combinations the phone keypad could represent. | backtrack one digit at a time, appending each candidate letter and undoing it. | O(4ⁿ·n) | O(n) | Standard |
| 22 | Generate Parentheses | Generate all combinations of n pairs of well-formed parentheses. | add `(` while opens remain; add `)` only while closes outstanding; backtrack each choice. | O(4ⁿ/√n) | O(n) | Standard |
| 36 | Valid Sudoku | Determine if a filled-in 9×9 Sudoku board is valid (no repeats in any row, column, or 3×3 box). | encode each digit's row/column/box membership into a HashSet of unique keys; a collision means invalid. | O(1) (fixed 81 cells) | O(1) | Standard |
| 77 | Combinations | Return all combinations of k numbers chosen from 1 to n. | standard backtracking with start index advancing to i+1; record when path reaches size k. | O(k·C(n,k)) | O(k) | Standard |
| 93 | Restore IP Addresses | Return all valid IP addresses formable by inserting dots into a digit string. | backtrack choosing 1-3 digit segments; each segment must be 0-255 and have no leading zero. | O(1) (bounded branching) | O(1) | Standard |
| 94 | Binary Tree Inorder Traversal | Return the inorder traversal (left → root → right) of a binary tree. | recurse left, visit node, recurse right. | O(n) | O(h) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 95 | Unique Binary Search Trees II | Generate all structurally unique BSTs storing values 1..n. | pick each value as root; recursively build all left subtrees from the smaller range and all right subtrees from the larger range, then combine. | O(n·Catalan(n)) | O(n·Catalan(n)) | Standard |
| 99 | Recover Binary Search Tree | Two nodes of a BST were swapped by mistake; recover the tree without changing its structure. | an inorder traversal of a valid BST is ascending; the swapped pair shows up as one or two descents. Track them and swap their values. | O(n) | O(h) | Standard |
| 100 | Same Tree | Check if two binary trees are structurally identical with the same node values. | both null is equal; one null or differing values is not; otherwise recurse on both children. | O(n) | O(h) | Standard |
| 101 | Symmetric Tree | Check if a binary tree is a mirror image of itself. | compare the outer pairs and inner pairs of two mirrored subtrees. | O(n) | O(h) | Standard |
| 102 | Binary Tree Level Order Traversal | Return node values level by level, left to right. | BFS with a queue; drain one level's worth of nodes per outer iteration. | O(n) | O(n) | Standard |
| 103 | Binary Tree Zigzag Level Order Traversal | Return node values level by level, alternating left-to-right and right-to-left. | standard BFS, but reverse the level list on alternate levels. | O(n) | O(n) | Standard |
| 105 | Construct Binary Tree from Preorder and Inorder Traversal | Reconstruct a binary tree from its preorder and inorder traversals. | the first preorder element is the root; its position in inorder splits left/right subtree sizes. | O(n) | O(n) | Standard |
| 106 | Construct Binary Tree from Inorder and Postorder Traversal | Reconstruct a binary tree from its inorder and postorder traversals. | the last postorder element is the root; build right subtree before left since postorder is consumed back to front. | O(n) | O(n) | Standard |
| 107 | Binary Tree Level Order Traversal II | Return node values level by level, from bottom to top. | standard BFS, then reverse the list of levels. | O(n) | O(n) | Standard |
| 108 | Convert Sorted Array to Binary Search Tree | Convert a sorted array to a height-balanced BST. | pick the middle element as the root so left and right halves are balanced; recurse on each half. | O(n) | O(log n) | Standard |
| 111 | Minimum Depth of Binary Tree | Find the shortest path length from root to any leaf. | like max depth, but a node with one missing child must take the existing child's depth (a non-leaf cannot end a root-to-leaf path). | O(n) | O(h) | skip the null side |
| 114 | Flatten Binary Tree to Linked List | Flatten a binary tree into a right-pointer linked list following preorder, in place. | reverse-preorder traversal (right, left, node) lets you prepend each node to a running tail. | O(n) | O(h) | Standard |
| 116 | Populating Next Right Pointers in Each Node | Connect each node's `next` pointer to its right neighbor in a perfect binary tree, using O(1) extra space. | use already-established `next` links of the current level to thread the next level. | O(n) | O(1) | Standard |
| 117 | Populating Next Right Pointers in Each Node II | Same as #116 but for an arbitrary binary tree. | walk each level via `next` links, building the next level's chain through a dummy head since children may be missing. | O(n) | O(1) | Standard |
| 144 | Binary Tree Preorder Traversal | Return the preorder traversal (root → left → right) of a binary tree. | visit node, recurse left, recurse right. | O(n) | O(h) | Standard |
| 145 | Binary Tree Postorder Traversal | Return the postorder traversal (left → right → root) of a binary tree. | recurse left, recurse right, visit node. | O(n) | O(h) | Standard |
| 199 | Binary Tree Right Side View | Return the values visible from the right side, one per level. | BFS each level; the last node dequeued in a level is the rightmost visible one. | O(n) | O(n) | Standard |
| 222 | Count Complete Tree Nodes | Count nodes in a complete binary tree faster than O(n). | if leftmost and rightmost depths match, the subtree is perfect (2^h − 1 nodes); otherwise recurse on both children. | O($\log^2 n$) | O(log n) | perfect subtree shortcut |
| 235 | Lowest Common Ancestor of a Binary Search Tree | Find the LCA of two nodes in a BST. | if both values are less than the node, go left; if both greater, go right; otherwise the node is the split point = LCA. | O(h) | O(1) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 236 | Lowest Common Ancestor of a Binary Tree | Find the LCA of two nodes in a general binary tree. | post-order: if p or q matches the node, return it; the node where both arms return non-null is the LCA. | O(n) | O(h) | Standard |
| 241 | Different Ways to Add Parentheses | Compute all results obtainable by parenthesizing an expression differently. | at each operator, split into left and right subexpressions, recursively evaluate both, and combine every pair. | O(Catalan(n)) | O(Catalan(n)) | Standard |
| 250 | Count Univalue Subtrees | Count subtrees in which all nodes share the same value. | post-order; a subtree is univalue if both children are univalue and their values match the node's value. | O(n) | O(h) | Standard |
| 255 | Verify Preorder Sequence in Binary Search Tree | Verify whether an array is a valid BST preorder traversal. | use a stack to simulate the path; popping when a value exceeds the top sets a lower bound. Any later value below that bound is invalid. | O(n) | O(n) | Standard |
| 267 | Palindrome Permutation II | Return all palindrome permutations of a string. | a palindrome needs at most one odd-count character; build half the palindrome by permuting half the characters, then mirror it. | O((n/2)!) | O(n) | Standard |
| 272 | Closest Binary Search Tree Value II | Find the k values in a BST closest to a target. | inorder gives sorted values; keep a sliding window of size k, evicting the farther end while a closer candidate exists. | O(n) | O(n) | values only get farther, stop early |
| 282 | Expression Add Operators | Insert +, -, * between digits of a string to reach a target value; return all expressions. | backtrack each operand prefix; track running value and the last multiplied term so * can fold correctly. | $O(4^n)$ | O(n) | Standard |
| 291 | Word Pattern II | Check if a string follows a pattern where each pattern char maps to a non-empty, non-overlapping substring. | backtrack assigning substrings to pattern characters, enforcing a bijection via two maps. | O(exponential) | O(n) | Standard |
| 293 | Flip Game | Given a string of '+' and '-', return all states after flipping one "++" to "--". | scan for each "++" occurrence and produce the flipped string. | $O(n^2)$ | O(n) | Standard |
| 294 | Flip Game II | Determine if the first player can guarantee a win in Flip Game. | the current player wins if any "++" flip leaves the opponent in a losing position; memoize states. | O(exponential, memoized) | O(states) | Standard |
| 297 | Serialize and Deserialize Binary Tree | Serialize a binary tree to a string and deserialize it back. | preorder with explicit "#" null markers uniquely encodes structure; rebuild by consuming tokens in the same order. | O(n) | O(n) | Standard |
| 298 | Binary Tree Longest Consecutive Sequence | Find the longest consecutive increasing-by-1 path going from parent to child. | pass the current run length down; extend it when the child is exactly parent + 1, otherwise reset to 1. | O(n) | O(h) | Standard |
| 301 | Remove Invalid Parentheses | Remove the minimum number of parentheses to make a string valid; return all distinct results. | BFS by removal count; the first level whose strings include valid ones is the minimum-removal answer. | $O(2^n)$ | $O(2^n)$ | Standard |
| 306 | Additive Number | Check if a string forms an additive sequence (each number is the sum of the two preceding). | backtrack the first two numbers; the rest of the string is forced, so just verify it matches the running sums. | $O(n^3)$ | O(n) | Standard |
| 314 | Binary Tree Vertical Order Traversal | Return node values ordered by column, then by row. | BFS tracking each node's column; collect values per column, then read columns left to right. | O(n) | O(n) | Standard |
| 320 | Generalized Abbreviation | Return all abbreviations of a word (replace any non-adjacent groups of letters with their counts). | for each character, choose to either abbreviate it (extend a count) or keep it literally; backtrack. | $O(2^n \cdot n)$ | O(n) | Standard |
| 339 | Nested List Weight Sum | Sum each integer in a nested list weighted by its depth. | DFS carrying the current depth; integers add `value·depth`, lists recurse one level deeper. | O(n) | O(d) | Standard |
| 364 | Nested List Weight Sum II | Sum each integer weighted by its inverse depth (deepest integers have weight 1). | weight = (maxDepth − depth + 1). Accumulate sum of values at each level and a running unweighted total per BFS level so deeper values get counted more often. | O(n) | O(n) | shallow values re-counted each level |
| 386 | Lexicographical Numbers | Return integers 1..n in lexicographical order. | DFS a 10-ary prefix tree: start at each digit, append 0-9 as long as the number stays ≤ n. | O(n) | O(log n) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 404 | Sum of Left Leaves | Sum the values of all left leaves in a binary tree. | pass down whether a node is a left child; add its value when it is both a left child and a leaf. | O(n) | O(h) | Standard |
| 426 | Convert Binary Search Tree to Sorted Doubly Linked List | Convert a BST into a sorted circular doubly linked list in place. | inorder traversal yields sorted order; link each node to the previous one, then close the ring between head and tail. | O(n) | O(h) | Standard |
| 429 | N-ary Tree Level Order Traversal | Return the level order traversal of an N-ary tree. | BFS, enqueuing all children of each node per level. | O(n) | O(n) | Standard |
| 437 | Path Sum III | Count downward paths (any node to any descendant) summing to a target. | prefix-sum the running root-to-node sum; the number of paths ending here equals how many earlier prefixes equal `current − target`. | O(n) | O(n) | prefix-sum count |
| 440 | K-th Smallest in Lexicographical Order | Find the kth smallest integer in 1..n by lexicographical order. | count how many numbers share a prefix; skip whole subtrees of the 10-ary prefix tree when k exceeds their size, otherwise descend. | $O(\log^2 n)$ | O(1) | Standard |
| 449 | Serialize and Deserialize BST | Serialize and deserialize a BST compactly. | preorder alone reconstructs a BST using value bounds, so no null markers are needed. | O(n) | O(n) | Standard |
| 450 | Delete Node in a BST | Delete a key from a BST and return the new root. | find the node; if it has two children, replace its value with the inorder successor (smallest in right subtree) and delete that successor. | O(h) | O(h) | Standard |
| 473 | Matchsticks to Square | Determine if matchsticks can form a square (4 equal sides) using all sticks. | backtrack placing each stick into one of 4 side buckets; prune when total isn't divisible by 4 or a stick exceeds the side length. | $O(4^n)$ | O(n) | skip equal buckets |
| 488 | Zuma Game | Find the minimum balls from hand to insert to clear the board (or -1). | DFS each board state, trying every hand ball at every insertion point, removing groups of 3+; memoize visited states. | O(exponential) | O(states) | Standard |
| 489 | Robot Room Cleaner | Clean an entire room with a robot that has only relative move/turn/clean APIs and no map. | DFS with backtracking using relative coordinates; clean, try all four directions, and reverse-move to return after each branch. | O(cells) | O(cells) | Standard |
| 491 | Increasing Subsequences | Return all increasing subsequences of length ≥ 2 (input may have duplicates). | backtrack choosing elements ≥ the last picked; within one recursion level use a set to skip duplicate starts. Cannot sort (would destroy original order). | $O(2^n \cdot n)$ | O(n) | skip dup at same level |
| 501 | Find Mode in Binary Search Tree | Find the mode(s) (most frequent values) in a BST that may have duplicates. | inorder gives sorted values, so equal values are adjacent; track current run count and update the mode list when counts tie or exceed the max. | O(n) | O(h) | Standard |
| 508 | Most Frequent Subtree Sum | Find the subtree sum(s) that occur most frequently. | post-order compute each subtree's sum, tally frequencies in a map, then collect the keys with the max frequency. | O(n) | O(n) | Standard |
| 510 | Inorder Successor in BST II | Find the inorder successor of a node given only a parent pointer (no root). | if a right subtree exists, the successor is its leftmost node; otherwise climb up until you come from a left child. | O(h) | O(1) | Standard |
| 513 | Find Bottom Left Tree Value | Find the leftmost value in the last (deepest) row of a binary tree. | BFS scanning right-to-left so the final node dequeued is the bottom-left value. | O(n) | O(n) | Standard |
| 515 | Find Largest Value in Each Tree Row | Return the maximum value in each row of a binary tree. | BFS level by level, tracking the max per level. | O(n) | O(n) | Standard |
| 526 | Beautiful Arrangement | Count permutations of 1..n where each number divides or is divisible by its position. | backtrack placing numbers position by position, only choosing a number when it satisfies the divisibility rule. | O(k) where k = valid arrangements | O(n) | Standard |
| 536 | Construct Binary Tree from String | Build a binary tree from a string like `4(2(3)(1))(6(5))`. | parse the leading number as the node, then the first parenthesized group as the left subtree and the second as the right subtree. | O(n) | O(h) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 538 | Convert BST to Greater Tree | Replace each node's value with the sum of all values greater than or equal to it. | reverse inorder (right → node → left) visits values in descending order; carry a running sum and add it into each node. | O(n) | O(h) | Standard |
| 545 | Boundary of Binary Tree | Return the boundary: root, left boundary top-down, leaves left-to-right, right boundary bottom-up, no duplicates. | collect three parts separately — left boundary (excluding leaves), all leaves, and right boundary reversed (excluding leaves). | O(n) | O(h) | Standard |
| 559 | Maximum Depth of N-ary Tree | Find the maximum depth of an N-ary tree. | depth is 1 plus the max depth among all children. | O(n) | O(h) | Standard |
| 563 | Binary Tree Tilt | Sum the tilt of every node, where tilt = \|left subtree sum − right subtree sum\|. | post-order return each subtree sum; accumulate the absolute difference into a global total. | O(n) | O(h) | Standard |
| 572 | Subtree of Another Tree | Check whether `subRoot` is a subtree of `root`. | at every node of `root`, test for structural equality with `subRoot`. | O(m·n) | O(h) | Standard |
| 589 | N-ary Tree Preorder Traversal | Return the preorder traversal of an N-ary tree. | visit the node, then recurse into each child in order. | O(n) | O(h) | Standard |
| 590 | N-ary Tree Postorder Traversal | Return the postorder traversal of an N-ary tree. | recurse into all children first, then visit the node. | O(n) | O(h) | Standard |
| 606 | Construct String from Binary Tree | Build a preorder string with parentheses, e.g. `1(2(4))(3)`. | append the value, then each non-null child wrapped in parentheses; keep empty `()` for a left child only when a right child exists. | O(n) | O(h) | Standard |
| 617 | Merge Two Binary Trees | Merge two binary trees by summing overlapping node values. | if either node is null, return the other; otherwise sum values and recurse on both child pairs. | O(n) | O(h) | Standard |
| 637 | Average of Levels in Binary Tree | Return the average value of nodes on each level. | BFS each level, summing values and dividing by the level size. | O(n) | O(n) | Standard |
| 652 | Find Duplicate Subtrees | Return one root per group of structurally identical, duplicated subtrees. | serialize each subtree post-order into a canonical string; the second time a serialization appears, record its root. | $O(n^2)$ | $O(n^2)$ | Standard |
| 653 | Two Sum IV - Input is a BST | Find whether two nodes in a BST sum to a target. | traverse, storing seen values in a set; for each node check if `target − val` was seen. | O(n) | O(n) | Standard |
| 654 | Maximum Binary Tree | Build a tree where the root is the array max, left subtree from the left subarray, right subtree from the right subarray. | find the max in the range as root, recurse on the two halves. | $O(n^2)$ | O(n) | Standard |
| 662 | Maximum Width of Binary Tree | Return the maximum width of any level, where width counts the gap between the leftmost and rightmost non-null nodes. | assign heap-style indices (left = 2i, right = 2i+1); per level the width is last index − first index + 1. | O(n) | O(n) | Standard |
| 671 | Second Minimum Node In a Binary Tree | In a tree where each node's value is the min of its children, find the second smallest value overall. | the root is the global minimum; DFS for the smallest value strictly greater than the root. | O(n) | O(h) | Standard |
| 679 | 24 Game | Determine if four numbers can be combined with +, −, ×, ÷ to make 24. | backtrack: pick any two numbers, apply each operator, replace them with the result, and recurse until one number remains. | O(1) (bounded) | O(1) | Standard |
| 698 | Partition to K Equal Sum Subsets | Determine if an array can be split into k subsets of equal sum. | target = total/k; backtrack filling one bucket at a time, sorting descending to fail fast, skipping when total isn't divisible. | $O(k \cdot 2^n)$ | O(n) | Standard |
| 700 | Search in a Binary Search Tree | Find the subtree rooted at the node with a given value in a BST. | binary-search the tree: go left if target is smaller, right if larger. | O(h) | O(1) | Standard |
| 701 | Insert into a Binary Search Tree | Insert a value into a BST and return the root. | descend like a search until reaching a null child, then attach a new node there. | O(h) | O(h) | Standard |
| 742 | Closest Leaf in a Binary Tree | Find the value of the leaf nearest to the node with a given value. | convert the tree to an undirected graph, then BFS from the target node until the first leaf is reached. | O(n) | O(n) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 776 | Split BST | Split a BST into two trees: one with values ≤ V and one with values > V. | recurse; at each node decide which result tree it belongs to based on V, then reattach the recursively-split child. | O(h) | O(h) | Standard |
| 784 | Letter Case Permutation | Return all strings formable by toggling the case of letters in a string. | backtrack character by character; digits have one choice, letters branch into lower and upper case. | $O(2^n \cdot n)$ | O(n) | Standard |
| 814 | Binary Tree Pruning | Remove every subtree that contains no node with value 1. | post-order prune children first; drop a node if both children become null and its own value is 0. | O(n) | O(h) | Standard |
| 842 | Split Array into Fibonacci Sequence | Split a digit string into a Fibonacci-like sequence (each term = sum of previous two), with terms fitting in a 32-bit int. | backtrack the first two numbers; the rest is forced. Prune leading zeros and overflow. | $O(n^2)$ | O(n) | Standard |
| 863 | All Nodes Distance K in Binary Tree | Find all node values exactly distance k from a target node. | build parent links so the tree becomes an undirected graph, then BFS k levels from the target. | O(n) | O(n) | Standard |
| 865 | Smallest Subtree with all the Deepest Nodes | Find the smallest subtree containing all of the tree's deepest leaves. | post-order return (depth, lca). If left and right depths tie, the current node is the LCA; otherwise carry up the deeper side. | O(n) | O(h) | tie → this node is LCA |
| 889 | Construct Binary Tree from Preorder and Postorder Traversal | Reconstruct a binary tree from preorder and postorder traversals (any valid tree). | preorder[start] is the root; preorder[start+1] is the left subtree root, whose position in postorder gives the left subtree size. | O(n) | O(h) | Standard |
| 897 | Increasing Order Search Tree | Rearrange a BST into a right-skewed tree following inorder order. | inorder traversal; relink each node as the right child of the previous, clearing left pointers. | O(n) | O(h) | Standard |
| 919 | Complete Binary Tree Inserter | Design a structure that supports O(1) insertion into a complete binary tree. | keep a BFS queue of nodes that still have a free child slot; insert into the front node and enqueue the new node. | O(1) insert | O(n) | Standard |
| 932 | Beautiful Array | Construct an array of 1..n where no three indices i<j<k satisfy 2·a[j] = a[i] + a[k]. | divide and conquer: a beautiful array of odds followed by evens stays beautiful, since odd + even is never twice an integer. | O(n log n) | O(n) | Standard |
| 938 | Range Sum of BST | Sum all node values within the inclusive range [low, high]. | prune branches outside the range using BST ordering. | O(n) | O(h) | Standard |
| 951 | Flip Equivalent Binary Trees | Check if two trees are flip-equivalent (can be made identical by swapping some children). | match children either directly or swapped at each node. | O(n) | O(h) | Standard |
| 958 | Check Completeness of a Binary Tree | Determine if a binary tree is complete. | BFS including null children; once a null is seen, no real node may follow. | O(n) | O(n) | real node after a gap |
| 987 | Vertical Order Traversal of a Binary Tree | Group node values by column, then by row, then by value within the same position. | DFS recording (col, row, val); sort by column, then row, then value. | O(n log n) | O(n) | Standard |
| 988 | Smallest String Starting From Leaf | Return the lexicographically smallest root-to-leaf string, where each node maps to a letter 'a'..'z' and the string is read leaf to root. | build the path down, reverse it at each leaf, and track the minimum. | O(n·h) | O(h) | Standard |
| 993 | Cousins in Binary Tree | Check if two values are cousins (same depth, different parents). | BFS recording each value's depth and parent; cousins share depth but differ in parent. | O(n) | O(n) | Standard |
| 998 | Maximum Binary Tree II | Insert a value into a maximum tree built by appending the value to the original array's end. | since the value was appended last, it lies on the rightmost spine; insert where it first exceeds the current node. | O(h) | O(h) | Standard |
| 1008 | Construct Binary Search Tree from Preorder Traversal | Build a BST from its preorder traversal. | the first value is the root; values less than it form the left subtree, the rest form the right. Use an upper bound to decide where to stop. | O(n) | O(h) | Standard |
| 1026 | Maximum Difference Between Node and Ancestor | Find the maximum \|a − d\| over all ancestor a and descendant d pairs. | pass the min and max seen on the path down; at each node update the answer with the largest gap against those extremes. | O(n) | O(h) | Standard |

| # | Name | Description | Intuition | Time | Space | Key Implementation |
|---|---|---|---|---|---|---|
| 1038 | Binary Search Tree to Greater Sum Tree | Replace each node's value with the sum of all values greater than or equal to it. | reverse inorder (right → node → left) processes descending order; carry a running sum. | O(n) | O(h) | Standard |
| 1087 | Brace Expansion | Expand a string with brace options like `{a,b}c{d,e}` into all sorted concatenations. | parse each position into a sorted list of choices, then backtrack the Cartesian product. | O(product of group sizes) | O(n) | Standard |
| 1104 | Path In Zigzag Labelled Binary Tree | Return the root-to-node path of labels in a zigzag-labeled infinite binary tree. | the normal parent of label is label/2, but rows alternate direction, so mirror the parent within its row range. | O(log n) | O(log n) | mirror then halve |
| 1110 | Delete Nodes And Return Forest | Delete the given values and return the roots of the resulting forest. | post-order; if a node is deleted, its surviving children become new roots. A node is a root if its parent was deleted (or it's the original root) and it itself survives. | O(n) | O(h) | Standard |
| 1123 | Lowest Common Ancestor of Deepest Leaves | Find the LCA of all the deepest leaves. | post-order return (depth, lca); when both arms reach equal depth, the node is the LCA, else carry up the deeper arm. | O(n) | O(h) | Standard |
| 1161 | Maximum Level Sum of a Binary Tree | Return the 1-indexed level with the largest sum of node values. | BFS level by level, tracking the level whose sum is maximum. | O(n) | O(n) | Standard |
| 1214 | Two Sum BSTs | Given two BSTs and a target, decide if a value from each sums to the target. | store all values of the first BST in a set, then traverse the second checking for the complement. | O(m + n) | O(m) | Standard |
| 1305 | All Elements in Two Binary Search Trees | Return all elements from two BSTs in sorted order. | inorder each tree into a sorted list, then merge the two lists like merge sort. | O(m + n) | O(m + n) | Standard |
| 1361 | Validate Binary Tree Nodes | Given child arrays, verify the nodes form exactly one valid binary tree. | exactly one node has no parent (the root); every other node has exactly one parent, there are no cycles, and all nodes are reachable from the root. | O(n) | O(n) | Standard |
| 1382 | Balance a Binary Search Tree | Rebuild a BST so it becomes height-balanced. | inorder gives a sorted array; build a balanced BST by recursively choosing the middle element as root. | O(n) | O(n) | Standard |
| 1522 | Diameter of N-Ary Tree | Find the diameter (longest path between any two nodes) of an N-ary tree. | like binary-tree diameter, but track the two deepest child heights to form the longest path through each node. | O(n) | O(h) | Standard |
| 1644 | Lowest Common Ancestor of a Binary Tree II | Find the LCA of two nodes that may not both exist in the tree; return null if either is missing. | standard LCA but count how many of p, q were actually found, so a node returned without seeing both isn't accepted. | O(n) | O(h) | Standard |
| 1650 | Lowest Common Ancestor of a Binary Tree III | Find the LCA of two nodes given parent pointers, without root access. | like finding the intersection of two linked lists — walk both pointers up, switching to the other node when one reaches the top; they meet at the LCA. | O(h) | O(1) | Standard |

## All Templates

```
// 1. PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)
// MENTAL MODEL: each node trusts its children to return a finished answer, then merges them.
// WHEN: "what is the height/sum/property OF the subtree rooted here"
private int dfs(TreeNode node) {
    if (node == null) return BASE;          // 0 for depth, true for balanced
    int left  = dfs(node.left);
    int right = dfs(node.right);
    // combine left, right, node.val → answer propagates up
    return ...;
}

// 2. PROCESS NODE FIRST, PASS STATE DOWN (pre-order)
// MENTAL MODEL: carry what you've seen so far down the path; the leaf has the full picture.
// WHEN: "root-to-leaf path", "running sum/number built along the way"
private void dfs(TreeNode node, int accumulated) {
    if (node == null) return;
    accumulated += node.val;                // update state on way down
    if (isLeaf(node)) { /* record */ return; }
    dfs(node.left,  accumulated);
    dfs(node.right, accumulated);
    // backtrack: no undo needed for primitive; undo List.add if using collection
}

// 3. TREE — GLOBAL ANSWER (return up the best single arm; update global across both arms)
// MENTAL MODEL: the answer may bend through a node using both arms, but only one arm can extend upward.
// WHEN: use when the best path spans across a node, not just up one arm.
int answer = Integer.MIN_VALUE;
private int dfs(TreeNode node) {
    if (node == null) return 0;
    int left  = Math.max(0, dfs(node.left));   // clamp negative
    int right = Math.max(0, dfs(node.right));
    answer = Math.max(answer, left + right + node.val);   // cross-node path
    return Math.max(left, right) + node.val;              // single arm up
}

// 4. GRID DFS — mark visited by mutating grid; 4-directional flood fill
// MENTAL MODEL: stand on a cell, flood into all 4 neighbors, sinking each as you visit it.
// WHEN: "connected region / island / flood fill" on a grid
private void dfs(int[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] != TARGET) return;
    grid[r][c] = VISITED;
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}

// 5. GRAPH DFS — 3-color visited: 0=unseen 1=in-stack 2=done
// MENTAL MODEL: if you re-enter a node still on the current stack, you've looped back → cycle.
// WHEN: "can finish all", "detect cycle", "valid ordering" on a directed graph
int[] state;
private boolean dfs(int node) {
    if (state[node] == 1) return true;    // back edge → cycle
    if (state[node] == 2) return false;   // already safe
    state[node] = 1;
    for (int neighbor : graph.get(node))
        if (dfs(neighbor)) return true;
    state[node] = 2;
    return false;
}

// 6. BACKTRACKING — choose / recurse / undo
// MENTAL MODEL: build a candidate one element at a time; undo each choice to explore the next branch.
// WHEN: "all subsets / permutations / combinations", "find all ways"
```

```
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition(i)) continue;    // prune duplicates
        current.add(nums[i]);              // choose
        backtrack(next, current);          // next = i (reuse) or i+1 (no reuse)
        current.remove(current.size()-1);  // undo
    }
}
```

---

# Part 1 — Tree DFS

## PROCESS CHILDREN FIRST, COMBINE AT NODE (post-order)

Recurse into children first, then combine results at the current node. Answer propagates upward.

### #104 Maximum Depth of Binary Tree

**Description:** Maximum number of nodes along the longest root-to-leaf path.
**Intuition:** my depth is just one more than my deeper child's depth.

```
class Solution {
    public int maxDepth(TreeNode root) {
        return dfs(root);
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left  = dfs(node.left);
        int right = dfs(node.right);
        return Math.max(left, right) + 1;
    }
}
```

**Time** O(n) | **Space** O(h)

---

### #543 Diameter of Binary Tree

**Description:** Length of the longest path between any two nodes (path does not need to pass through root).
**Key:** the path through a node = `left height + right height`. Track global max; return only the single best arm up.
**Intuition:** the longest path bends at some node using both arms, but only one arm can extend to its parent.

```
class Solution {
    int diameter = 0;
    public int diameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return diameter;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left  = dfs(node.left);
        int right = dfs(node.right);
        diameter = Math.max(diameter, left + right);   // cross-node path
        return Math.max(left, right) + 1;              // single arm up
    }
}
```

**Time** O(n) | **Space** O(h)

---

### #124 Binary Tree Maximum Path Sum

**Description:** Maximum sum of any path between any two nodes. Values can be negative.
**Key:** clamp negative arms to 0 ( `Math.max(0, ...)` ). Global tracks the best `left + right + node.val` ; return only `max(left, right) +`

`node.val` upward.
**Intuition:** a negative arm is worse than taking nothing, so clamp it to 0 before adding.

```
class Solution {
    int maxSum = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) {
        dfs(root);
        return maxSum;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left  = Math.max(0, dfs(node.left));
        int right = Math.max(0, dfs(node.right));
        maxSum = Math.max(maxSum, left + right + node.val);
        return Math.max(left, right) + node.val;
    }
}
```

**Time** O(n) | **Space** O(h)

---

### #110 Balanced Binary Tree

**Description:** Is every node's left and right subtree within 1 height of each other?
**Key:** return -1 as a sentinel for "unbalanced"; short-circuit immediately on -1.
**Intuition:** fold "is it balanced" into the height return — a poisoned -1 bubbles up and stops the work.

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return dfs(root) != -1;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left  = dfs(node.left);
        if (left  == -1) return -1;
        int right = dfs(node.right);
        if (right == -1) return -1;
        if (Math.abs(left - right) > 1) return -1;
        return Math.max(left, right) + 1;
    }
}
```

**Time** O(n) | **Space** O(h)

---

### #226 Invert Binary Tree

**Description:** Mirror a binary tree — swap left and right subtrees at every node.
**Intuition:** invert both children, then swap the pointers at this node.

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;
        TreeNode left  = invertTree(root.left);
        TreeNode right = invertTree(root.right);
        root.left  = right;
        root.right = left;
        return root;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## PROCESS NODE FIRST, PASS STATE DOWN (pre-order)

State is built on the way **down** and consumed at leaves. Use backtracking when state is a mutable collection.

### #112 Path Sum

**Description:** Does any root-to-leaf path sum to `targetSum`?
**Intuition:** subtract each node from the target on the way down; a leaf with remaining 0 is a hit.

```
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        return dfs(root, targetSum);
    }
    private boolean dfs(TreeNode node, int remaining) {
        if (node == null) return false;
        remaining -= node.val;
        if (node.left == null && node.right == null) return remaining == 0;
        return dfs(node.left, remaining) || dfs(node.right, remaining);
    }
}
```

**Time** O(n) | **Space** O(h)

---

### #113 Path Sum II

**Description:** Return all root-to-leaf paths that sum to `targetSum`.
**Key:** pass a mutable `path` list down. **Backtrack** (remove last element) after returning from each child.
**Intuition:** the same `path` list is reused for every branch, so undo your add as you unwind.

```
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> result = new ArrayList<>();
        dfs(root, targetSum, new ArrayList<>(), result);
        return result;
    }
    private void dfs(TreeNode node, int remaining, List<Integer> path, List<List<Integer>> result) {
        if (node == null) return;
        path.add(node.val);
        remaining -= node.val;
        if (node.left == null && node.right == null && remaining == 0)
            result.add(new ArrayList<>(path));
        dfs(node.left,  remaining, path, result);
        dfs(node.right, remaining, path, result);
        path.remove(path.size() - 1);   // backtrack
    }
}
```

**Time** O(n) | **Space** O(n) for output paths

---

### #257 Binary Tree Paths

**Description:** Return all root-to-leaf paths as strings ( `"1->2->5"` ).
**Intuition:** append to a shared StringBuilder, then truncate back to your entry length to undo.

```java
class Solution {
    public List<String> binaryTreePaths(TreeNode root) {
        List<String> result = new ArrayList<>();
        dfs(root, new StringBuilder(), result);
        return result;
    }
    private void dfs(TreeNode node, StringBuilder path, List<String> result) {
        if (node == null) return;
        int len = path.length();
        if (len > 0) path.append("->");
        path.append(node.val);
        if (node.left == null && node.right == null)
            result.add(path.toString());
        dfs(node.left,  path, result);
        dfs(node.right, path, result);
        path.setLength(len);   // backtrack to before this node
    }
}
```

**Time** O(n) | **Space** O(n) for output paths

---

### #129 Sum Root to Leaf Numbers

**Description:** Each root-to-leaf path represents a number (e.g., 1→2→3 = 123). Return the total sum.
**Intuition:** build the number digit by digit on the way down ( `current*10 + val` ); add it up at each leaf.

```java
class Solution {
    public int sumNumbers(TreeNode root) {
        return dfs(root, 0);
    }
    private int dfs(TreeNode node, int current) {
        if (node == null) return 0;
        current = current * 10 + node.val;
        if (node.left == null && node.right == null) return current;
        return dfs(node.left, current) + dfs(node.right, current);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## 543 vs 124 — Side by Side

Both use the global-variable pattern. One difference:

```
                   #543 Diameter             #124 Max Path Sum
Global tracks:     left + right              left + right + node.val
Clamp negatives?   No (heights >= 0)         Yes (Math.max(0, ...))
Return up:         max(left, right) + 1      max(left, right) + node.val
Base case:         return 0                  return 0
```

---

# Part 2 — Graph DFS

## Grid DFS (4-directional flood fill)

Mark visited by overwriting the cell value. Return the accumulated property (count, area) or void.

### #200 Number of Islands

**Description:** Count the number of islands (connected groups of `'1'` ) in a binary grid.
**Intuition:** each unvisited land cell starts a new island; flood-fill sinks the whole island so it's counted once.

```
class Solution {
    public int numIslands(char[][] grid) {
        int count = 0;
        for (int r = 0; r < grid.length; r++)
            for (int c = 0; c < grid[0].length; c++)
                if (grid[r][c] == '1') { dfs(grid, r, c); count++; }
        return count;
    }
    private void dfs(char[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
        if (grid[r][c] != '1') return;
        grid[r][c] = '0';   // mark visited
        dfs(grid, r+1, c); dfs(grid, r-1, c);
        dfs(grid, r, c+1); dfs(grid, r, c-1);
    }
}
```

**Time** O(m·n) | **Space** O(m·n)

---

### #695 Max Area of Island

**Description:** Return the area of the largest island.
**Key:** same flood fill but DFS returns the area of the connected component.
**Intuition:** an island's area is 1 (this cell) plus the areas its four neighbors flood back.

```
class Solution {
    public int maxAreaOfIsland(int[][] grid) {
        int max = 0;
        for (int r = 0; r < grid.length; r++)
            for (int c = 0; c < grid[0].length; c++)
                max = Math.max(max, dfs(grid, r, c));
        return max;
    }
    private int dfs(int[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return 0;
        if (grid[r][c] != 1) return 0;
        grid[r][c] = 0;
        return 1 + dfs(grid, r+1, c) + dfs(grid, r-1, c)
                 + dfs(grid, r, c+1) + dfs(grid, r, c-1);
    }
}
```

**Time** O(m·n) | **Space** O(m·n)

---

## Graph DFS — 3-Color Cycle Detection

Three states: `0` = unvisited, `1` = in current DFS stack (visiting), `2` = fully processed (safe).
A back edge (reaching a node with state `1`) means a cycle.

### #207 Course Schedule

**Description:** Can you finish all courses given prerequisites? Return false if there is a cycle.
**Intuition:** re-entering a node still on the current DFS stack means you looped back to a prerequisite → cycle.

```
class Solution {
    int[] state;
    List<List<Integer>> graph;

    public boolean canFinish(int numCourses, int[][] prerequisites) {
        state = new int[numCourses];
        graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) graph.get(p[1]).add(p[0]);
        for (int i = 0; i < numCourses; i++)
            if (hasCycle(i)) return false;
        return true;
    }
    private boolean hasCycle(int node) {
        if (state[node] == 1) return true;    // back edge → cycle
        if (state[node] == 2) return false;   // already safe
        state[node] = 1;
        for (int neighbor : graph.get(node))
            if (hasCycle(neighbor)) return true;
        state[node] = 2;
        return false;
    }
}
```

**Time** O(V+E) | **Space** O(V+E)

---

### #210 Course Schedule II

**Description:** Return a valid order to take all courses. Return `[]` if a cycle exists.
**Key:** same 3-color DFS. Add node to `order` in **post-order** (after all neighbors done). Reverse at the end → topological order.
**Intuition:** a node finishes only after all it depends on do, so post-order reversed is a valid prerequisite order.

```
class Solution {
    int[] state;
    List<List<Integer>> graph;
    List<Integer> order;

    public int[] findOrder(int numCourses, int[][] prerequisites) {
        state = new int[numCourses];
        order = new ArrayList<>();
        graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) graph.get(p[1]).add(p[0]);
        for (int i = 0; i < numCourses; i++)
            if (!dfs(i)) return new int[]{};
        Collections.reverse(order);
        return order.stream().mapToInt(Integer::intValue).toArray();
    }
    private boolean dfs(int node) {
        if (state[node] == 1) return false;   // cycle
        if (state[node] == 2) return true;    // already done
        state[node] = 1;
        for (int neighbor : graph.get(node))
            if (!dfs(neighbor)) return false;
        state[node] = 2;
        order.add(node);                      // post-order
        return true;
    }
}
```

**Time** O(V+E) | **Space** O(V+E)

---

### 207 vs 210 — Side by Side

```
                  #207 (boolean)            #210 (order)
DFS returns:      boolean (cycle found?)    boolean (cycle found?)
Extra action:     —                         order.add(node)  after neighbors
Post-process:     —                         Collections.reverse(order)
```

# Part 3 — Backtracking

**Template:** choose → recurse → undo.
`start` controls which elements are eligible at each level. `current` is the in-progress candidate.

```
// MENTAL MODEL: grow one candidate by picking an element, recurse, then undo to try the next branch.
private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (skipCondition) continue;
        current.add(nums[i]);               // choose
        backtrack(nextStart, current);      // nextStart = i (reuse) or i+1 (no reuse)
        current.remove(current.size() - 1); // undo
    }
}
```

## Backtracking Decision Table

| # | All orderings? | Reuse element? | Has duplicates? | Start index | Skip condition |
|---|---|---|---|---|---|
| 46 Permutations | Yes — `used[]` | No | No | Loop 0..n, `used[i]` | `used[i]` |
| 78 Subsets | No | No | No | `start` → `i+1` | None |
| 39 Combination Sum | No | Yes | No | `start` → `i` | `candidates[i] > remaining` |
| 40 Combination Sum II | No | No | Yes | `start` → `i+1` | `i > start && nums[i] == nums[i-1]` |
| 131 Palindrome Part. | No | No | No | `start` → `end` | `!isPalindrome(sub)` |

### #46 Permutations

**Description:** All permutations of distinct integers.
**Key:** loop always from 0; use `boolean[] used` to avoid reusing elements within one permutation.
**Intuition:** order matters, so every position considers all unused elements (no `start` index).

```
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, boolean[] used, List<Integer> current, List<List<Integer>> result) {
        if (current.size() == nums.length) { result.add(new ArrayList<>(current)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            current.add(nums[i]);
            backtrack(nums, used, current, result);
            current.remove(current.size() - 1);
            used[i] = false;
        }
    }
}
```

**Time** O(n·n!) | **Space** O(n)

### #78 Subsets

**Description:** All subsets (power set) of distinct integers.
**Key:** record at every node (not just leaves). Pass `i+1` — each element used at most once, order enforced by `start`.
**Intuition:** every prefix on the way down is itself a valid subset, so record at every node.

```
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
        result.add(new ArrayList<>(current));   // record every prefix (including empty)
        for (int i = start; i < nums.length; i++) {
            current.add(nums[i]);
            backtrack(nums, i + 1, current, result);
            current.remove(current.size() - 1);
        }
    }
}
```

**Time** $O(n \cdot 2^n)$ | **Space** $O(n)$

---

### #39 Combination Sum

**Description:** All combinations summing to target. Same element may be used unlimited times. No duplicate combos.
**Key:** pass `i` (not `i+1`) to allow reuse. Sort + prune when `candidates[i] > remaining`.
**Intuition:** passing `i` (not `i+1`) lets you pick the same element again; `start` still forbids going backward, so no reordered dups.

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> result = new ArrayList<>();
        backtrack(candidates, 0, target, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] candidates, int start, int remaining, List<Integer> current, List<List<Integer>> result) {
        if (remaining == 0) { result.add(new ArrayList<>(current)); return; }
        for (int i = start; i < candidates.length; i++) {
            if (candidates[i] > remaining) break;   // pruning
            current.add(candidates[i]);
            backtrack(candidates, i, remaining - candidates[i], current, result);  // i, not i+1
            current.remove(current.size() - 1);
        }
    }
}
```

**Time** $O(n \cdot 2^n)$ | **Space** $O(target/min)$

---

### #40 Combination Sum II

**Description:** Each element used at most once. Input may have duplicates. No duplicate combos in output.
**Key:** sort first. Skip `candidates[i] == candidates[i-1]` when `i > start` — avoids duplicate combos at the same recursion level. Pass `i+1` — no reuse.
**Intuition:** after sorting, equal values sit together; skipping a repeat at the *same level* prevents producing the same combo twice.

```java
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> result = new ArrayList<>();
        backtrack(candidates, 0, target, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] candidates, int start, int remaining, List<Integer> current, List<List<Integer>> result) {
        if (remaining == 0) { result.add(new ArrayList<>(current)); return; }
        for (int i = start; i < candidates.length; i++) {
            if (candidates[i] > remaining) break;
            if (i > start && candidates[i] == candidates[i - 1]) continue;  // skip dup at same level
            current.add(candidates[i]);
            backtrack(candidates, i + 1, remaining - candidates[i], current, result);  // i+1, no reuse
            current.remove(current.size() - 1);
        }
    }
}
```

**Time** O(n · $2^n$) | **Space** O(target/min)

---

### #79 Word Search

**Description:** Given a board of characters, return true if the word exists as a connected path (no cell reused).
**Key:** mark cell as visited by overwriting, then restore after recursion (backtrack).
**Intuition:** the grid itself is the state — temporarily blank a cell so the same path can't reuse it, then restore on the way back.

```java
class Solution {
    public boolean exist(char[][] board, String word) {
        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++)
                if (dfs(board, word, r, c, 0)) return true;
        return false;
    }
    private boolean dfs(char[][] board, String word, int r, int c, int idx) {
        if (idx == word.length()) return true;
        if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
        if (board[r][c] != word.charAt(idx)) return false;
        char temp = board[r][c];
        board[r][c] = '#';   // mark visited
        boolean found = dfs(board, word, r+1, c, idx+1) || dfs(board, word, r-1, c, idx+1)
                     || dfs(board, word, r, c+1, idx+1) || dfs(board, word, r, c-1, idx+1);
        board[r][c] = temp;  // restore (backtrack)
        return found;
    }
}
```

**Time** O(m·n·4^L) where L = word length | **Space** O(L)

---

### #131 Palindrome Partitioning

**Description:** Partition string `s` such that every substring is a palindrome. Return all such partitions.
**Intuition:** try every prefix as the first cut; only recurse on the rest when that prefix is a palindrome.

```
class Solution {
    public List<List<String>> partition(String s) {
        List<List<String>> result = new ArrayList<>();
        backtrack(s, 0, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(String s, int start, List<String> current, List<List<String>> result) {
        if (start == s.length()) { result.add(new ArrayList<>(current)); return; }
        for (int end = start + 1; end <= s.length(); end++) {
            String sub = s.substring(start, end);
            if (!isPalindrome(sub)) continue;
            current.add(sub);
            backtrack(s, end, current, result);
            current.remove(current.size() - 1);
        }
    }
    private boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
        return true;
    }
}
```

**Time** O(n · 2ⁿ) | **Space** O(n)

---

### 39 vs 40 — The One Difference

```
                   #39 (reuse OK)                 #40 (no reuse, has dups)
Sort:              Yes (pruning)                  Yes (both pruning and dup-skip)
Recurse with:      i     (same index)             i+1   (next index)
Skip dup line:     —                              if (i > start && nums[i] == nums[i-1]) continue
```

### Tree DFS Summary

```
Pattern            When to use                        Answer location
Return up          Combine children at node           return value bubbles up
Pass down          Accumulate state toward leaves     check at leaf
Global variable    Path spans across a node (not up)  int/long field, updated inside dfs
```

---

# Part — BST (Binary Search Tree) Problems

BST key property: `left subtree values < node.val < right subtree values`. This allows O(h) search/insert/delete by binary-searching the tree.

### BST Template: Pass Bounds Down

```
// Validate BST by passing min/max constraints down the tree
boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;   // ← bounds check
    return validate(node.left, min, node.val)               // left: tighten max
        && validate(node.right, node.val, max);             // right: tighten min
}

// Call with: validate(root, Long.MIN_VALUE, Long.MAX_VALUE)
// Use Long to handle Integer.MIN_VALUE / Integer.MAX_VALUE edge cases
```

## BST Template: Inorder Traversal = Sorted Order

```
// BST inorder (left → node → right) visits nodes in ascending order
void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    // process node (kth smallest: increment counter, return when k)
    inorder(node.right);
}
```

## BST Template: Binary Search on Tree

```
// Navigate BST like binary search: go left or right based on comparison
TreeNode search(TreeNode root, int target) {
    while (root != null) {
        if (root.val == target) {
            return root;
        } else if (root.val < target) {
            root = root.right;
        } else {
            root = root.left;
        }
    }
    return null;
}
```

---

## #98 Validate Binary Search Tree

**Description:** Determine if a binary tree is a valid BST. Each node's value must be strictly greater than all values in its left subtree and strictly less than all values in its right subtree.

**Variation:** Pass min/max bounds down — each call tightens the valid range. Use `long` to handle `Integer.MIN_VALUE` and `Integer.MAX_VALUE` edge cases.

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;  // ← VARIATION: bounds constraint
        return validate(node.left, min, node.val)              // ← VARIATION: tighten max going left
            && validate(node.right, node.val, max);            // ← VARIATION: tighten min going right
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #230 Kth Smallest Element in a BST

**Description:** Find the kth smallest element in a BST (1-indexed).

**Variation:** BST inorder traversal visits nodes in ascending order. Count nodes visited; when count reaches k, record the answer and stop early.

```
class Solution {
    private int count = 0, result = 0;

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }

    private void inorder(TreeNode node, int k) {
        if (node == null) return;
        inorder(node.left, k);
        if (++count == k) { result = node.val; return; }  // ← VARIATION: stop at kth node
        inorder(node.right, k);
    }
}
```

**Time** O(k) average, O(n) worst | **Space** O(h)

---

## #270 Closest Binary Search Tree Value

**Description:** Given a BST and a `target` (double), return the value in the BST closest to target.

**Variation:** Use BST structure to navigate in O(h). At each node, compare distance to closest seen so far, then binary-search left or right.

```
class Solution {
    public int closestValue(TreeNode root, double target) {
        int closest = root.val;
        while (root != null) {
            if (Math.abs(root.val - target) < Math.abs(closest - target))
                closest = root.val;                      // ← VARIATION: track closest during search
            root = root.val < target ? root.right : root.left;  // ← VARIATION: binary search
        }
        return closest;
    }
}
```

**Time** O(h) | **Space** O(1)

---

## #285 Inorder Successor in BST

**Description:** Given a node `p` in a BST, return its inorder successor (the smallest node with value greater than `p.val`). Return null if no such node exists.

**Variation:** Navigate the BST. When `root.val > p.val`, this root is a candidate — save it and go left (to find a smaller valid candidate). When `root.val <= p.val`, go right (must find something larger).

```
class Solution {
    public TreeNode inorderSuccessor(TreeNode root, TreeNode p) {
        TreeNode result = null;
        while (root != null) {
            if (root.val > p.val) {
                result = root;          // ← VARIATION: save as candidate, then search left for closer
                root = root.left;
            } else {
                root = root.right;      // ← VARIATION: must go right to find something > p
            }
        }
        return result;
    }
}
```

**Time** O(h) | **Space** O(1)

---

## #687 Longest Univalue Path

**Description:** Return the length of the longest path in a binary tree where every node along the path has the same value. The path does not need to pass through the root.

**Variation:** Post-order DFS. At each node, extend the left or right path only if the child's value matches. The path through the current node = leftPath + rightPath. Return the max single-direction extension upward.

```
class Solution {
    private int max = 0;

    public int longestUnivaluePath(TreeNode root) {
        dfs(root);
        return max;
    }

    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left  = dfs(node.left);
        int right = dfs(node.right);
        int leftPath  = (node.left  != null && node.left.val  == node.val) ? left  + 1 : 0;  // ← VARIATION: only extend if values m
        int rightPath = (node.right != null && node.right.val == node.val) ? right + 1 : 0;  // ← VARIATION: conditional extension
        max = Math.max(max, leftPath + rightPath);   // ← path through this node
        return Math.max(leftPath, rightPath);         // ← return max single direction up
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #37 Sudoku Solver

**Description:** Fill a partially filled 9×9 Sudoku so every row, column, and 3×3 box contains digits 1-9. **Variation:** backtracking that tries digits 1-9 in each empty cell, validating against row, column, and box constraints before recursing.

```
class Solution {
    public void solveSudoku(char[][] board) { solve(board); }
    private boolean solve(char[][] board) {
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 9; j++)
                if (board[i][j] == '.') {
                    for (char c = '1'; c <= '9'; c++) {
                        if (isValid(board, i, j, c)) {     // ← VARIATION: validate row/col/box
                            board[i][j] = c;
                            if (solve(board)) return true;
                            board[i][j] = '.';             // backtrack
                        }
                    }
                    return false;                          // no digit fits → dead end
                }
        return true;                                       // all cells filled
    }
    private boolean isValid(char[][] board, int row, int col, char c) {
        for (int i = 0; i < 9; i++) {
            if (board[row][i] == c) return false;          // row
            if (board[i][col] == c) return false;          // column
            if (board[3*(row/3) + i/3][3*(col/3) + i%3] == c) return false;  // ← VARIATION: 3x3 box
        }
        return true;
    }
}
```

**Time** O(9^m) m=empty cells | **Space** O(1)

## #47 Permutations II

**Description:** Return all unique permutations of an array that may contain duplicates. **Variation:** sort first; use a `used[]` array; skip a value if the previous equal value at the same tree level has not been used (prevents duplicate permutations).

```
class Solution {
    public List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums);                              // ← VARIATION: sort to group duplicates
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> result) {
        if (path.size() == nums.length) { result.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue;  // ← VARIATION: skip duplicate at same level
            used[i] = true; path.add(nums[i]);
            backtrack(nums, used, path, result);
            used[i] = false; path.remove(path.size() - 1);
        }
    }
}
```

**Time** O(n·n!) | **Space** O(n)

## #51 N-Queens

**Description:** Place n queens on an n×n board so no two attack each other; return all distinct solutions. **Variation:** backtrack row by row; track occupied columns and both diagonals. Cells on the same `\` diagonal share `r-c`; cells on the same `/` diagonal share `r+c`.

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> result = new ArrayList<>();
        Set<Integer> cols = new HashSet<>(), diag1 = new HashSet<>(), diag2 = new HashSet<>();
        backtrack(0, n, new int[n], cols, diag1, diag2, result);
        return result;
    }
    private void backtrack(int row, int n, int[] queens, Set<Integer> cols,
                           Set<Integer> diag1, Set<Integer> diag2, List<List<String>> result) {
        if (row == n) { result.add(build(queens, n)); return; }
        for (int c = 0; c < n; c++) {
            if (cols.contains(c) || diag1.contains(row - c) || diag2.contains(row + c)) continue;  // ← VARIATION: diagonal sets
            queens[row] = c;
            cols.add(c); diag1.add(row - c); diag2.add(row + c);
            backtrack(row + 1, n, queens, cols, diag1, diag2, result);
            cols.remove(c); diag1.remove(row - c); diag2.remove(row + c);
        }
    }
    private List<String> build(int[] queens, int n) {
        List<String> board = new ArrayList<>();
        for (int r = 0; r < n; r++) {
            char[] row = new char[n];
            Arrays.fill(row, '.');
            row[queens[r]] = 'Q';
            board.add(new String(row));
        }
        return board;
    }
}
```

**Time** O(n!) | **Space** O(n)

## #60 Permutation Sequence

**Description:** Return the kth permutation (1-indexed) of the sequence 1..n. **Variation:** no backtracking — use the factorial number system. Each position's digit is determined directly by `(k-1) / (remaining-1)!`.

```
class Solution {
    public String getPermutation(int n, int k) {
        List<Integer> digits = new ArrayList<>();
        int[] factorial = new int[n + 1];
        factorial[0] = 1;
        for (int i = 1; i <= n; i++) { factorial[i] = factorial[i-1] * i; digits.add(i); }
        k--;                                            // ← VARIATION: convert to 0-indexed
        StringBuilder result = new StringBuilder();
        for (int i = n; i >= 1; i--) {
            int index = k / factorial[i-1];             // ← VARIATION: direct digit selection
            k %= factorial[i-1];
            result.append(digits.remove(index));
        }
        return result.toString();
    }
}
```

**Time** $O(n^2)$ | **Space** O(n)

## #90 Subsets II

**Description:** Return all possible unique subsets of an array that may contain duplicates. **Variation:** sort first; within a recursion level, skip a value equal to its predecessor to avoid duplicate subsets.

```
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);                              // ← VARIATION: sort to group duplicates
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>> result) {
        result.add(new ArrayList<>(path));
        for (int i = start; i < nums.length; i++) {
            if (i > start && nums[i] == nums[i-1]) continue;  // ← VARIATION: skip duplicate at same depth
            path.add(nums[i]);
            backtrack(nums, i + 1, path, result);
            path.remove(path.size() - 1);
        }
    }
}
```

**Time** $O(n \cdot 2^n)$ | **Space** O(n)

## #216 Combination Sum III

**Description:** Find all combinations of k numbers from 1-9 (each used at most once) that sum to n. **Variation:** standard combination backtracking with start index `i+1` (no reuse), but with two stopping constraints — count and remaining sum.

```
class Solution {
    public List<List<Integer>> combinationSum3(int k, int n) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(k, n, 1, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int k, int remain, int start, List<Integer> path, List<List<Integer>> result) {
        if (path.size() == k && remain == 0) { result.add(new ArrayList<>(path)); return; }  // ← VARIATION: two constraints
        if (path.size() == k || remain < 0) return;
        for (int i = start; i <= 9; i++) {
            path.add(i);
            backtrack(k, remain - i, i + 1, path, result);  // i+1: no reuse
            path.remove(path.size() - 1);
        }
    }
}
```

**Time** O(C(9,k)) | **Space** O(k)

### #549 Binary Tree Longest Consecutive Sequence II

**Description:** Find the length of the longest consecutive path in a binary tree. The path can be increasing or decreasing and may go through a node (child-parent-child). **Variation:** post-order DFS returns a pair `{increasing, decreasing}` for each node. A path through the node combines an increasing run from one child with a decreasing run from the other.

```
class Solution {
    private int max = 0;
    public int longestConsecutive(TreeNode root) {
        dfs(root);
        return max;
    }
    private int[] dfs(TreeNode node) {                      // returns {inc, dec}
        if (node == null) return new int[]{0, 0};
        int inc = 1, dec = 1;
        int[] left = dfs(node.left), right = dfs(node.right);
        if (node.left != null) {
            if (node.left.val + 1 == node.val) inc = left[0] + 1;   // ← VARIATION: increasing run
            else if (node.left.val - 1 == node.val) dec = left[1] + 1;  // ← VARIATION: decreasing run
        }
        if (node.right != null) {
            if (node.right.val + 1 == node.val) inc = Math.max(inc, right[0] + 1);
            else if (node.right.val - 1 == node.val) dec = Math.max(dec, right[1] + 1);
        }
        max = Math.max(max, inc + dec - 1);                 // ← VARIATION: combine both directions through node
        return new int[]{inc, dec};
    }
}
```

**Time** O(n) | **Space** O(h)

---

# Additional Reference Problems

### #17 Letter Combinations of a Phone Number

**Description:** Given a string of digits 2-9, return all letter combinations the phone keypad could represent. **Intuition:** backtrack one digit at a time, appending each candidate letter and undoing it.

```
class Solution {
    private static final String[] MAP = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    public List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();
        if (digits.isEmpty()) return result;
        backtrack(digits, 0, new StringBuilder(), result);
        return result;
    }
    private void backtrack(String digits, int start, StringBuilder builder, List<String> result) {
        if (start == digits.length()) { result.add(builder.toString()); return; }
        String letters = MAP[digits.charAt(start) - '0'];
        for (int i = 0; i < letters.length(); i++) {
            builder.append(letters.charAt(i));
            backtrack(digits, start + 1, builder, result);
            builder.deleteCharAt(builder.length() - 1);
        }
    }
}
```

**Time** $O(4^n \cdot n)$ | **Space** O(n)

---

## #22 Generate Parentheses

**Description:** Generate all combinations of n pairs of well-formed parentheses. **Intuition:** add `(` while opens remain; add `)` only while closes outstanding; backtrack each choice.

```
class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> result = new ArrayList<>();
        backtrack(n, n, new StringBuilder(), result);
        return result;
    }
    private void backtrack(int open, int close, StringBuilder builder, List<String> result) {
        if (open == 0 && close == 0) { result.add(builder.toString()); return; }
        if (open > 0) {
            builder.append('(');
            backtrack(open - 1, close, builder, result);
            builder.deleteCharAt(builder.length() - 1);
        }
        if (close > open) {
            builder.append(')');
            backtrack(open, close - 1, builder, result);
            builder.deleteCharAt(builder.length() - 1);
        }
    }
}
```

**Time** $O(4^n/\sqrt{n})$ | **Space** O(n)

---

## #36 Valid Sudoku

**Description:** Determine if a filled-in 9×9 Sudoku board is valid (no repeats in any row, column, or 3×3 box). **Intuition:** encode each digit's row/column/box membership into a HashSet of unique keys; a collision means invalid.

```
class Solution {
    public boolean isValidSudoku(char[][] board) {
        Set<String> seen = new HashSet<>();
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                char c = board[i][j];
                if (c == '.') { continue; }
                String row = c + "r" + i;
                String col = c + "c" + j;
                String box = c + "b" + (i / 3) + (j / 3);
                if (!seen.add(row) || !seen.add(col) || !seen.add(box)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

**Time** O(1) (fixed 81 cells) | **Space** O(1)

---

## #77 Combinations

**Description:** Return all combinations of k numbers chosen from 1 to n. **Intuition:** standard backtracking with start index advancing to i+1; record when path reaches size k.

```
class Solution {
    public List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(n, k, 1, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int n, int k, int start, List<Integer> path, List<List<Integer>> result) {
        if (path.size() == k) { result.add(new ArrayList<>(path)); return; }
        for (int i = start; i <= n; i++) {
            path.add(i);
            backtrack(n, k, i + 1, path, result);
            path.remove(path.size() - 1);
        }
    }
}
```

**Time** O(k·C(n,k)) | **Space** O(k)

---

## #93 Restore IP Addresses

**Description:** Return all valid IP addresses formable by inserting dots into a digit string. **Intuition:** backtrack choosing 1-3 digit segments; each segment must be 0-255 and have no leading zero.

```
class Solution {
    public List<String> restoreIpAddresses(String s) {
        List<String> result = new ArrayList<>();
        backtrack(s, 0, 0, new StringBuilder(), result);
        return result;
    }
    private void backtrack(String s, int start, int parts, StringBuilder builder, List<String> result) {
        if (parts == 4) {
            if (start == s.length()) { result.add(builder.substring(0, builder.length() - 1)); }
            return;
        }
        for (int len = 1; len <= 3 && start + len <= s.length(); len++) {
            String segment = s.substring(start, start + len);
            if (segment.length() > 1 && segment.charAt(0) == '0') { break; }
            if (Integer.parseInt(segment) > 255) { break; }
            int mark = builder.length();
            builder.append(segment).append('.');
            backtrack(s, start + len, parts + 1, builder, result);
            builder.setLength(mark);
        }
    }
}
```

**Time** O(1) (bounded branching) | **Space** O(1)

---

## #94 Binary Tree Inorder Traversal

**Description:** Return the inorder traversal (left → root → right) of a binary tree. **Intuition:** recurse left, visit node, recurse right.

```
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        dfs(root, result);
        return result;
    }
    private void dfs(TreeNode node, List<Integer> result) {
        if (node == null) return;
        dfs(node.left, result);
        result.add(node.val);
        dfs(node.right, result);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #95 Unique Binary Search Trees II

**Description:** Generate all structurally unique BSTs storing values 1..n. **Intuition:** pick each value as root; recursively build all left subtrees from the smaller range and all right subtrees from the larger range, then combine.

```
class Solution {
    public List<TreeNode> generateTrees(int n) {
        if (n == 0) return new ArrayList<>();
        return build(1, n);
    }
    private List<TreeNode> build(int lo, int hi) {
        List<TreeNode> result = new ArrayList<>();
        if (lo > hi) { result.add(null); return result; }
        for (int i = lo; i <= hi; i++) {
            List<TreeNode> left = build(lo, i - 1);
            List<TreeNode> right = build(i + 1, hi);
            for (TreeNode l : left) {
                for (TreeNode r : right) {
                    TreeNode root = new TreeNode(i);
                    root.left = l;
                    root.right = r;
                    result.add(root);
                }
            }
        }
        return result;
    }
}
```

**Time** O(n·Catalan(n)) | **Space** O(n·Catalan(n))

---

## #99 Recover Binary Search Tree

**Description:** Two nodes of a BST were swapped by mistake; recover the tree without changing its structure. **Intuition:** an inorder traversal of a valid BST is ascending; the swapped pair shows up as one or two descents. Track them and swap their values.

```
class Solution {
    private TreeNode first = null, second = null, prev = null;
    public void recoverTree(TreeNode root) {
        inorder(root);
        int temp = first.val;
        first.val = second.val;
        second.val = temp;
    }
    private void inorder(TreeNode node) {
        if (node == null) return;
        inorder(node.left);
        if (prev != null && prev.val > node.val) {
            if (first == null) { first = prev; }
            second = node;
        }
        prev = node;
        inorder(node.right);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #100 Same Tree

**Description:** Check if two binary trees are structurally identical with the same node values. **Intuition:** both null is equal; one null or differing values is not; otherwise recurse on both children.

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) return true;
        if (p == null || q == null) return false;
        if (p.val != q.val) return false;
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #101 Symmetric Tree

**Description:** Check if a binary tree is a mirror image of itself. **Intuition:** compare the outer pairs and inner pairs of two mirrored subtrees.

```
class Solution {
    public boolean isSymmetric(TreeNode root) {
        if (root == null) return true;
        return mirror(root.left, root.right);
    }
    private boolean mirror(TreeNode a, TreeNode b) {
        if (a == null && b == null) return true;
        if (a == null || b == null) return false;
        if (a.val != b.val) return false;
        return mirror(a.left, b.right) && mirror(a.right, b.left);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #102 Binary Tree Level Order Traversal

**Description:** Return node values level by level, left to right. **Intuition:** BFS with a queue; drain one level's worth of nodes per outer iteration.

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            result.add(level);
        }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #103 Binary Tree Zigzag Level Order Traversal

**Description:** Return node values level by level, alternating left-to-right and right-to-left. **Intuition:** standard BFS, but reverse the level list on alternate levels.

```
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        boolean leftToRight = true;
        while (!queue.isEmpty()) {
            int size = queue.size();
            LinkedList<Integer> level = new LinkedList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (leftToRight) { level.addLast(node.val); }
                else { level.addFirst(node.val); }
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            result.add(level);
            leftToRight = !leftToRight;
        }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #105 Construct Binary Tree from Preorder and Inorder Traversal

**Description:** Reconstruct a binary tree from its preorder and inorder traversals. **Intuition:** the first preorder element is the root; its position in inorder splits left/right subtree sizes.

```
class Solution {
    private int preIndex = 0;
    private Map<Integer, Integer> inIndex = new HashMap<>();
    private int[] preorder;
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        this.preorder = preorder;
        for (int i = 0; i < inorder.length; i++) { inIndex.put(inorder[i], i); }
        return build(0, inorder.length - 1);
    }
    private TreeNode build(int lo, int hi) {
        if (lo > hi) return null;
        int rootVal = preorder[preIndex++];
        TreeNode root = new TreeNode(rootVal);
        int mid = inIndex.get(rootVal);
        root.left = build(lo, mid - 1);
        root.right = build(mid + 1, hi);
        return root;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #106 Construct Binary Tree from Inorder and Postorder Traversal

**Description:** Reconstruct a binary tree from its inorder and postorder traversals. **Intuition:** the last postorder element is the root; build right subtree before left since postorder is consumed back to front.

```
class Solution {
    private int postIndex;
    private Map<Integer, Integer> inIndex = new HashMap<>();
    private int[] postorder;
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        this.postorder = postorder;
        this.postIndex = postorder.length - 1;
        for (int i = 0; i < inorder.length; i++) { inIndex.put(inorder[i], i); }
        return build(0, inorder.length - 1);
    }
    private TreeNode build(int lo, int hi) {
        if (lo > hi) return null;
        int rootVal = postorder[postIndex--];
        TreeNode root = new TreeNode(rootVal);
        int mid = inIndex.get(rootVal);
        root.right = build(mid + 1, hi);
        root.left = build(lo, mid - 1);
        return root;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #107 Binary Tree Level Order Traversal II

**Description:** Return node values level by level, from bottom to top. **Intuition:** standard BFS, then reverse the list of levels.

```
class Solution {
    public List<List<Integer>> levelOrderBottom(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            result.add(level);
        }
        Collections.reverse(result);
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #108 Convert Sorted Array to Binary Search Tree

**Description:** Convert a sorted array to a height-balanced BST. **Intuition:** pick the middle element as the root so left and right halves are balanced; recurse on each half.

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return build(nums, 0, nums.length - 1);
    }
    private TreeNode build(int[] nums, int lo, int hi) {
        if (lo > hi) return null;
        int mid = lo + (hi - lo) / 2;
        TreeNode root = new TreeNode(nums[mid]);
        root.left = build(nums, lo, mid - 1);
        root.right = build(nums, mid + 1, hi);
        return root;
    }
}
```

**Time** O(n) | **Space** O(log n)

---

## #111 Minimum Depth of Binary Tree

**Description:** Find the shortest path length from root to any leaf. **Intuition:** like max depth, but a node with one missing child must take the existing child's depth (a non-leaf cannot end a root-to-leaf path).

```
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null) return 0;
        int left = minDepth(root.left);
        int right = minDepth(root.right);
        if (root.left == null || root.right == null) {
            return Math.max(left, right) + 1;   // ← VARIATION: skip the null side
        }
        return Math.min(left, right) + 1;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #114 Flatten Binary Tree to Linked List

**Description:** Flatten a binary tree into a right-pointer linked list following preorder, in place. **Intuition:** reverse-preorder traversal (right, left, node) lets you prepend each node to a running tail.

```
class Solution {
    private TreeNode prev = null;
    public void flatten(TreeNode root) {
        if (root == null) return;
        flatten(root.right);
        flatten(root.left);
        root.right = prev;
        root.left = null;
        prev = root;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #116 Populating Next Right Pointers in Each Node

**Description:** Connect each node's `next` pointer to its right neighbor in a perfect binary tree, using O(1) extra space. **Intuition:** use already-established `next` links of the current level to thread the next level.

```
class Solution {
    public Node connect(Node root) {
        Node leftmost = root;
        while (leftmost != null && leftmost.left != null) {
            Node head = leftmost;
            while (head != null) {
                head.left.next = head.right;
                if (head.next != null) { head.right.next = head.next.left; }
                head = head.next;
            }
            leftmost = leftmost.left;
        }
        return root;
    }
}
```

**Time** O(n) | **Space** O(1)

---

## #117 Populating Next Right Pointers in Each Node II

**Description:** Same as #116 but for an arbitrary binary tree. **Intuition:** walk each level via `next` links, building the next level's chain through a dummy head since children may be missing.

```
class Solution {
    public Node connect(Node root) {
        Node head = root;
        while (head != null) {
            Node dummy = new Node(0);
            Node tail = dummy;
            for (Node cur = head; cur != null; cur = cur.next) {
                if (cur.left != null) { tail.next = cur.left; tail = tail.next; }
                if (cur.right != null) { tail.next = cur.right; tail = tail.next; }
            }
            head = dummy.next;
        }
        return root;
    }
}
```

**Time** O(n) | **Space** O(1)

---

## #144 Binary Tree Preorder Traversal

**Description:** Return the preorder traversal (root → left → right) of a binary tree. **Intuition:** visit node, recurse left, recurse right.

```
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        dfs(root, result);
        return result;
    }
    private void dfs(TreeNode node, List<Integer> result) {
        if (node == null) return;
        result.add(node.val);
        dfs(node.left, result);
        dfs(node.right, result);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #145 Binary Tree Postorder Traversal

**Description:** Return the postorder traversal (left → right → root) of a binary tree. **Intuition:** recurse left, recurse right, visit node.

```
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        dfs(root, result);
        return result;
    }
    private void dfs(TreeNode node, List<Integer> result) {
        if (node == null) return;
        dfs(node.left, result);
        dfs(node.right, result);
        result.add(node.val);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #199 Binary Tree Right Side View

**Description:** Return the values visible from the right side, one per level. **Intuition:** BFS each level; the last node dequeued in a level is the rightmost visible one.

```
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (i == size - 1) { result.add(node.val); }
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
        }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #222 Count Complete Tree Nodes

**Description:** Count nodes in a complete binary tree faster than O(n). **Intuition:** if leftmost and rightmost depths match, the subtree is perfect (2^h – 1 nodes); otherwise recurse on both children.

```
class Solution {
    public int countNodes(TreeNode root) {
        if (root == null) return 0;
        int left = leftDepth(root);
        int right = rightDepth(root);
        if (left == right) { return (1 << left) - 1; }   // ← VARIATION: perfect subtree shortcut
        return 1 + countNodes(root.left) + countNodes(root.right);
    }
    private int leftDepth(TreeNode node) {
        int d = 0;
        while (node != null) { d++; node = node.left; }
        return d;
    }
    private int rightDepth(TreeNode node) {
        int d = 0;
        while (node != null) { d++; node = node.right; }
        return d;
    }
}
```

**Time** O(log²n) | **Space** O(log n)

---

## #235 Lowest Common Ancestor of a Binary Search Tree

**Description:** Find the LCA of two nodes in a BST. **Intuition:** if both values are less than the node, go left; if both greater, go right; otherwise the node is the split point = LCA.

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val) { root = root.left; }
            else if (p.val > root.val && q.val > root.val) { root = root.right; }
            else { return root; }
        }
        return null;
    }
}
```

**Time** O(h) | **Space** O(1)

---

## #236 Lowest Common Ancestor of a Binary Tree

**Description:** Find the LCA of two nodes in a general binary tree. **Intuition:** post-order: if p or q matches the node, return it; the node where both arms return non-null is the LCA.

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) return root;
        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);
        if (left != null && right != null) return root;
        return left != null ? left : right;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #241 Different Ways to Add Parentheses

**Description:** Compute all results obtainable by parenthesizing an expression differently. **Intuition:** at each operator, split into left and right subexpressions, recursively evaluate both, and combine every pair.

```
class Solution {
    public List<Integer> diffWaysToCompute(String expression) {
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < expression.length(); i++) {
            char c = expression.charAt(i);
            if (c == '+' || c == '-' || c == '*') {
                List<Integer> left = diffWaysToCompute(expression.substring(0, i));
                List<Integer> right = diffWaysToCompute(expression.substring(i + 1));
                for (int a : left) {
                    for (int b : right) {
                        if (c == '+') { result.add(a + b); }
                        else if (c == '-') { result.add(a - b); }
                        else { result.add(a * b); }
                    }
                }
            }
        }
        if (result.isEmpty()) { result.add(Integer.parseInt(expression)); }
        return result;
    }
}
```

**Time** O(Catalan(n)) | **Space** O(Catalan(n))

---

## #250 Count Univalue Subtrees

**Description:** Count subtrees in which all nodes share the same value. **Intuition:** post-order; a subtree is univalue if both children are univalue and their values match the node's value.

```
class Solution {
    private int count = 0;
    public int countUnivalSubtrees(TreeNode root) {
        isUnival(root);
        return count;
    }
    private boolean isUnival(TreeNode node) {
        if (node == null) return true;
        boolean left = isUnival(node.left);
        boolean right = isUnival(node.right);
        if (!left || !right) return false;
        if (node.left != null && node.left.val != node.val) return false;
        if (node.right != null && node.right.val != node.val) return false;
        count++;
        return true;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #255 Verify Preorder Sequence in Binary Search Tree

**Description:** Verify whether an array is a valid BST preorder traversal. **Intuition:** use a stack to simulate the path; popping when a value exceeds the top sets a lower bound. Any later value below that bound is invalid.

```
class Solution {
    public boolean verifyPreorder(int[] preorder) {
        Deque<Integer> stack = new ArrayDeque<>();
        int lowerBound = Integer.MIN_VALUE;
        for (int value : preorder) {
            if (value < lowerBound) return false;
            while (!stack.isEmpty() && stack.peek() < value) {
                lowerBound = stack.pop();
            }
            stack.push(value);
        }
        return true;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #267 Palindrome Permutation II

**Description:** Return all palindrome permutations of a string. **Intuition:** a palindrome needs at most one odd-count character; build half the palindrome by permuting half the characters, then mirror it.

```
class Solution {
    public List<String> generatePalindromes(String s) {
        List<String> result = new ArrayList<>();
        int[] count = new int[128];
        for (char c : s.toCharArray()) { count[c]++; }
        String mid = "";
        List<Character> half = new ArrayList<>();
        for (int i = 0; i < 128; i++) {
            if (count[i] % 2 == 1) {
                if (!mid.isEmpty()) return result;   // more than one odd → impossible
                mid = String.valueOf((char) i);
            }
            for (int j = 0; j < count[i] / 2; j++) { half.add((char) i); }
        }
        backtrack(half, new boolean[half.size()], new StringBuilder(), mid, result);
        return result;
    }
    private void backtrack(List<Character> half, boolean[] used, StringBuilder builder, String mid, List<String> result) {
        if (builder.length() == half.size()) {
            String left = builder.toString();
            result.add(left + mid + new StringBuilder(left).reverse());
            return;
        }
        for (int i = 0; i < half.size(); i++) {
            if (used[i]) continue;
            if (i > 0 && half.get(i).equals(half.get(i - 1)) && !used[i - 1]) continue;
            used[i] = true;
            builder.append(half.get(i));
            backtrack(half, used, builder, mid, result);
            builder.deleteCharAt(builder.length() - 1);
            used[i] = false;
        }
    }
}
```

**Time** O((n/2)!) | **Space** O(n)

---

## #272 Closest Binary Search Tree Value II

**Description:** Find the k values in a BST closest to a target. **Intuition:** inorder gives sorted values; keep a sliding window of size k, evicting the farther end while a closer candidate exists.

```
class Solution {
    public List<Integer> closestKValues(TreeNode root, double target, int k) {
        Deque<Integer> window = new ArrayDeque<>();
        inorder(root, target, k, window);
        return new ArrayList<>(window);
    }
    private void inorder(TreeNode node, double target, int k, Deque<Integer> window) {
        if (node == null) return;
        inorder(node.left, target, k, window);
        if (window.size() < k) {
            window.offerLast(node.val);
        } else if (Math.abs(node.val - target) < Math.abs(window.peekFirst() - target)) {
            window.pollFirst();
            window.offerLast(node.val);
        } else {
            return;   // ← VARIATION: values only get farther, stop early
        }
        inorder(node.right, target, k, window);
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #282 Expression Add Operators

**Description:** Insert +, -, * between digits of a string to reach a target value; return all expressions. **Intuition:** backtrack each operand prefix; track running value and the last multiplied term so * can fold correctly.

```
class Solution {
    public List<String> addOperators(String num, int target) {
        List<String> result = new ArrayList<>();
        backtrack(num, target, 0, 0, 0, new StringBuilder(), result);
        return result;
    }
    private void backtrack(String num, int target, int start, long value, long prev, StringBuilder builder, List<String> result) {
        if (start == num.length()) {
            if (value == target) { result.add(builder.toString()); }
            return;
        }
        for (int i = start; i < num.length(); i++) {
            if (i > start && num.charAt(start) == '0') break;   // no leading zero
            long cur = Long.parseLong(num.substring(start, i + 1));
            int len = builder.length();
            if (start == 0) {
                builder.append(cur);
                backtrack(num, target, i + 1, cur, cur, builder, result);
                builder.setLength(len);
            } else {
                builder.append('+').append(cur);
                backtrack(num, target, i + 1, value + cur, cur, builder, result);
                builder.setLength(len);

                builder.append('-').append(cur);
                backtrack(num, target, i + 1, value - cur, -cur, builder, result);
                builder.setLength(len);

                builder.append('*').append(cur);
                backtrack(num, target, i + 1, value - prev + prev * cur, prev * cur, builder, result);
                builder.setLength(len);
            }
        }
    }
}
```

**Time** O($4^n$) | **Space** O(n)

---

## #291 Word Pattern II

**Description:** Check if a string follows a pattern where each pattern char maps to a non-empty, non-overlapping substring. **Intuition:** backtrack assigning substrings to pattern characters, enforcing a bijection via two maps.

```
class Solution {
    public boolean wordPatternMatch(String pattern, String s) {
        return backtrack(pattern, 0, s, 0, new HashMap<>(), new HashSet<>());
    }
    private boolean backtrack(String pattern, int pi, String s, int si, Map<Character, String> map, Set<String> used) {
        if (pi == pattern.length() && si == s.length()) return true;
        if (pi == pattern.length() || si == s.length()) return false;
        char c = pattern.charAt(pi);
        if (map.containsKey(c)) {
            String mapped = map.get(c);
            if (!s.startsWith(mapped, si)) return false;
            return backtrack(pattern, pi + 1, s, si + mapped.length(), map, used);
        }
        for (int i = si; i < s.length(); i++) {
            String candidate = s.substring(si, i + 1);
            if (used.contains(candidate)) continue;
            map.put(c, candidate);
            used.add(candidate);
            if (backtrack(pattern, pi + 1, s, i + 1, map, used)) return true;
            map.remove(c);
            used.remove(candidate);
        }
        return false;
    }
}
```

**Time** O(exponential) | **Space** O(n)

---

## #293 Flip Game

**Description:** Given a string of '+' and '-', return all states after flipping one "++" to "--". **Intuition:** scan for each "++" occurrence and produce the flipped string.

```
class Solution {
    public List<String> generatePossibleNextMoves(String currentState) {
        List<String> result = new ArrayList<>();
        char[] arr = currentState.toCharArray();
        for (int i = 0; i + 1 < arr.length; i++) {
            if (arr[i] == '+' && arr[i + 1] == '+') {
                arr[i] = '-';
                arr[i + 1] = '-';
                result.add(new String(arr));
                arr[i] = '+';
                arr[i + 1] = '+';
            }
        }
        return result;
    }
}
```

**Time** O($n^2$) | **Space** O(n)

---

## #294 Flip Game II

**Description:** Determine if the first player can guarantee a win in Flip Game. **Intuition:** the current player wins if any "++" flip leaves the opponent in a losing position; memoize states.

```
class Solution {
    public boolean canWin(String currentState) {
        return canWin(currentState, new HashMap<>());
    }
    private boolean canWin(String state, Map<String, Boolean> memo) {
        if (memo.containsKey(state)) return memo.get(state);
        for (int i = 0; i + 1 < state.length(); i++) {
            if (state.charAt(i) == '+' && state.charAt(i + 1) == '+') {
                String next = state.substring(0, i) + "--" + state.substring(i + 2);
                if (!canWin(next, memo)) {
                    memo.put(state, true);
                    return true;
                }
            }
        }
        memo.put(state, false);
        return false;
    }
}
```

**Time** O(exponential, memoized) | **Space** O(states)

---

## #297 Serialize and Deserialize Binary Tree

**Description:** Serialize a binary tree to a string and deserialize it back. **Intuition:** preorder with explicit "#" null markers uniquely encodes structure; rebuild by consuming tokens in the same order.

```
public class Codec {
    public String serialize(TreeNode root) {
        StringBuilder builder = new StringBuilder();
        serialize(root, builder);
        return builder.toString();
    }
    private void serialize(TreeNode node, StringBuilder builder) {
        if (node == null) { builder.append("#,"); return; }
        builder.append(node.val).append(',');
        serialize(node.left, builder);
        serialize(node.right, builder);
    }
    public TreeNode deserialize(String data) {
        Queue<String> queue = new ArrayDeque<>(Arrays.asList(data.split(",")));
        return deserialize(queue);
    }
    private TreeNode deserialize(Queue<String> queue) {
        String token = queue.poll();
        if (token.equals("#")) return null;
        TreeNode node = new TreeNode(Integer.parseInt(token));
        node.left = deserialize(queue);
        node.right = deserialize(queue);
        return node;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #298 Binary Tree Longest Consecutive Sequence

**Description:** Find the longest consecutive increasing-by-1 path going from parent to child. **Intuition:** pass the current run length down; extend it when the child is exactly parent + 1, otherwise reset to 1.

```
class Solution {
    private int max = 0;
    public int longestConsecutive(TreeNode root) {
        dfs(root, null, 0);
        return max;
    }
    private void dfs(TreeNode node, TreeNode parent, int length) {
        if (node == null) return;
        if (parent != null && node.val == parent.val + 1) { length++; }
        else { length = 1; }
        max = Math.max(max, length);
        dfs(node.left, node, length);
        dfs(node.right, node, length);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #301 Remove Invalid Parentheses

**Description:** Remove the minimum number of parentheses to make a string valid; return all distinct results. **Intuition:** BFS by removal count; the first level whose strings include valid ones is the minimum-removal answer.

```
class Solution {
    public List<String> removeInvalidParentheses(String s) {
        List<String> result = new ArrayList<>();
        Set<String> visited = new HashSet<>();
        Queue<String> queue = new ArrayDeque<>();
        queue.offer(s);
        visited.add(s);
        boolean found = false;
        while (!queue.isEmpty()) {
            String cur = queue.poll();
            if (isValid(cur)) { result.add(cur); found = true; }
            if (found) continue;
            for (int i = 0; i < cur.length(); i++) {
                if (cur.charAt(i) != '(' && cur.charAt(i) != ')') continue;
                String next = cur.substring(0, i) + cur.substring(i + 1);
                if (visited.add(next)) { queue.offer(next); }
            }
        }
        return result;
    }
    private boolean isValid(String s) {
        int balance = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') { balance++; }
            else if (c == ')') { balance--; if (balance < 0) return false; }
        }
        return balance == 0;
    }
}
```

**Time** $O(2^n)$ | **Space** $O(2^n)$

---

## #306 Additive Number

**Description:** Check if a string forms an additive sequence (each number is the sum of the two preceding). **Intuition:** backtrack the first two numbers; the rest of the string is forced, so just verify it matches the running sums.

```
class Solution {
    public boolean isAdditiveNumber(String num) {
        int n = num.length();
        for (int i = 1; i <= n / 2; i++) {
            if (num.charAt(0) == '0' && i > 1) break;
            for (int j = i + 1; n - j >= Math.max(i, j - i); j++) {
                if (num.charAt(i) == '0' && j - i > 1) break;
                long first = Long.parseLong(num.substring(0, i));
                long second = Long.parseLong(num.substring(i, j));
                if (check(first, second, j, num)) return true;
            }
        }
        return false;
    }
    private boolean check(long first, long second, int start, String num) {
        if (start == num.length()) return true;
        long sum = first + second;
        String sumStr = Long.toString(sum);
        if (!num.startsWith(sumStr, start)) return false;
        return check(second, sum, start + sumStr.length(), num);
    }
}
```

**Time** O(n³) | **Space** O(n)

---

## #314 Binary Tree Vertical Order Traversal

**Description:** Return node values ordered by column, then by row. **Intuition:** BFS tracking each node's column; collect values per column, then read columns left to right.

```
class Solution {
    public List<List<Integer>> verticalOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Map<Integer, List<Integer>> columns = new HashMap<>();
        Queue<TreeNode> queue = new ArrayDeque<>();
        Queue<Integer> cols = new ArrayDeque<>();
        queue.offer(root);
        cols.offer(0);
        int min = 0, max = 0;
        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            int col = cols.poll();
            columns.computeIfAbsent(col, k -> new ArrayList<>()).add(node.val);
            min = Math.min(min, col);
            max = Math.max(max, col);
            if (node.left != null) { queue.offer(node.left); cols.offer(col - 1); }
            if (node.right != null) { queue.offer(node.right); cols.offer(col + 1); }
        }
        for (int i = min; i <= max; i++) { result.add(columns.get(i)); }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #320 Generalized Abbreviation

**Description:** Return all abbreviations of a word (replace any non-adjacent groups of letters with their counts). **Intuition:** for each character, choose to either abbreviate it (extend a count) or keep it literally; backtrack.

```
class Solution {
    public List<String> generateAbbreviations(String word) {
        List<String> result = new ArrayList<>();
        backtrack(word, 0, 0, new StringBuilder(), result);
        return result;
    }
    private void backtrack(String word, int start, int count, StringBuilder builder, List<String> result) {
        if (start == word.length()) {
            int len = builder.length();
            if (count > 0) { builder.append(count); }
            result.add(builder.toString());
            builder.setLength(len);
            return;
        }
        backtrack(word, start + 1, count + 1, builder, result);   // abbreviate this char
        int len = builder.length();
        if (count > 0) { builder.append(count); }
        builder.append(word.charAt(start));
        backtrack(word, start + 1, 0, builder, result);            // keep this char
        builder.setLength(len);
    }
}
```

**Time** O($2^n \cdot n$) | **Space** O(n)

---

## #339 Nested List Weight Sum

**Description:** Sum each integer in a nested list weighted by its depth. **Intuition:** DFS carrying the current depth; integers add `value·depth`, lists recurse one level deeper.

```
class Solution {
    public int depthSum(List<NestedInteger> nestedList) {
        return dfs(nestedList, 1);
    }
    private int dfs(List<NestedInteger> list, int depth) {
        int sum = 0;
        for (NestedInteger ni : list) {
            if (ni.isInteger()) { sum += ni.getInteger() * depth; }
            else { sum += dfs(ni.getList(), depth + 1); }
        }
        return sum;
    }
}
```

**Time** O(n) | **Space** O(d)

---

## #364 Nested List Weight Sum II

**Description:** Sum each integer weighted by its inverse depth (deepest integers have weight 1). **Intuition:** weight = (maxDepth − depth + 1). Accumulate sum of values at each level and a running unweighted total per BFS level so deeper values get counted more often.

```
class Solution {
    public int depthSumInverse(List<NestedInteger> nestedList) {
        int unweighted = 0, total = 0;
        Queue<NestedInteger> queue = new ArrayDeque<>(nestedList);
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                NestedInteger ni = queue.poll();
                if (ni.isInteger()) { unweighted += ni.getInteger(); }
                else { queue.addAll(ni.getList()); }
            }
            total += unweighted;   // ← VARIATION: shallow values re-counted each level
        }
        return total;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #386 Lexicographical Numbers

**Description:** Return integers 1..n in lexicographical order. **Intuition:** DFS a 10-ary prefix tree: start at each digit, append 0-9 as long as the number stays ≤ n.

```
class Solution {
    public List<Integer> lexicalOrder(int n) {
        List<Integer> result = new ArrayList<>();
        for (int i = 1; i <= 9; i++) { dfs(i, n, result); }
        return result;
    }
    private void dfs(int cur, int n, List<Integer> result) {
        if (cur > n) return;
        result.add(cur);
        for (int i = 0; i <= 9; i++) { dfs(cur * 10 + i, n, result); }
    }
}
```

**Time** O(n) | **Space** O(log n)

---

## #404 Sum of Left Leaves

**Description:** Sum the values of all left leaves in a binary tree. **Intuition:** pass down whether a node is a left child; add its value when it is both a left child and a leaf.

```
class Solution {
    public int sumOfLeftLeaves(TreeNode root) {
        return dfs(root, false);
    }
    private int dfs(TreeNode node, boolean isLeft) {
        if (node == null) return 0;
        if (node.left == null && node.right == null) {
            return isLeft ? node.val : 0;
        }
        return dfs(node.left, true) + dfs(node.right, false);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #426 Convert Binary Search Tree to Sorted Doubly Linked List

**Description:** Convert a BST into a sorted circular doubly linked list in place. **Intuition:** inorder traversal yields sorted order; link each node to the previous one, then close the ring between head and tail.

```
class Solution {
    private Node first = null, last = null;
    public Node treeToDoublyList(Node root) {
        if (root == null) return null;
        inorder(root);
        last.right = first;
        first.left = last;
        return first;
    }
    private void inorder(Node node) {
        if (node == null) return;
        inorder(node.left);
        if (last != null) { last.right = node; node.left = last; }
        else { first = node; }
        last = node;
        inorder(node.right);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #429 N-ary Tree Level Order Traversal

**Description:** Return the level order traversal of an N-ary tree. **Intuition:** BFS, enqueuing all children of each node per level.

```
class Solution {
    public List<List<Integer>> levelOrder(Node root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;
        Queue<Node> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                Node node = queue.poll();
                level.add(node.val);
                for (Node child : node.children) { queue.offer(child); }
            }
            result.add(level);
        }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #437 Path Sum III

**Description:** Count downward paths (any node to any descendant) summing to a target. **Intuition:** prefix-sum the running root-to-node sum; the number of paths ending here equals how many earlier prefixes equal `current - target`.

```
class Solution {
    public int pathSum(TreeNode root, int targetSum) {
        Map<Long, Integer> prefix = new HashMap<>();
        prefix.put(0L, 1);
        return dfs(root, 0L, targetSum, prefix);
    }
    private int dfs(TreeNode node, long current, int target, Map<Long, Integer> prefix) {
        if (node == null) return 0;
        current += node.val;
        int count = prefix.getOrDefault(current - target, 0);   // ← VARIATION: prefix-sum count
        prefix.merge(current, 1, Integer::sum);
        count += dfs(node.left, current, target, prefix);
        count += dfs(node.right, current, target, prefix);
        prefix.merge(current, -1, Integer::sum);                // backtrack
        return count;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #440 K-th Smallest in Lexicographical Order

**Description:** Find the kth smallest integer in 1..n by lexicographical order. **Intuition:** count how many numbers share a prefix; skip whole subtrees of the 10-ary prefix tree when k exceeds their size, otherwise descend.

```
class Solution {
    public int findKthNumber(int n, int k) {
        long cur = 1;
        k--;
        while (k > 0) {
            long steps = countSteps(n, cur, cur + 1);
            if (steps <= k) {
                cur++;
                k -= steps;
            } else {
                cur *= 10;
                k--;
            }
        }
        return (int) cur;
    }
    private long countSteps(int n, long first, long last) {
        long steps = 0;
        while (first <= n) {
            steps += Math.min((long) n + 1, last) - first;
            first *= 10;
            last *= 10;
        }
        return steps;
    }
}
```

**Time** $O(\log^2 n)$ | **Space** O(1)

---

## #449 Serialize and Deserialize BST

**Description:** Serialize and deserialize a BST compactly. **Intuition:** preorder alone reconstructs a BST using value bounds, so no null markers are needed.

```
public class Codec {
    public String serialize(TreeNode root) {
        StringBuilder builder = new StringBuilder();
        serialize(root, builder);
        return builder.toString();
    }
    private void serialize(TreeNode node, StringBuilder builder) {
        if (node == null) return;
        builder.append(node.val).append(',');
        serialize(node.left, builder);
        serialize(node.right, builder);
    }
    private int index = 0;
    private String[] tokens;
    public TreeNode deserialize(String data) {
        if (data.isEmpty()) return null;
        tokens = data.split(",");
        index = 0;
        return build(Integer.MIN_VALUE, Integer.MAX_VALUE);
    }
    private TreeNode build(int min, int max) {
        if (index == tokens.length) return null;
        int val = Integer.parseInt(tokens[index]);
        if (val < min || val > max) return null;
        index++;
        TreeNode node = new TreeNode(val);
        node.left = build(min, val);
        node.right = build(val, max);
        return node;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #450 Delete Node in a BST

**Description:** Delete a key from a BST and return the new root. **Intuition:** find the node; if it has two children, replace its value with the inorder successor (smallest in right subtree) and delete that successor.

```
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;
        if (key < root.val) {
            root.left = deleteNode(root.left, key);
        } else if (key > root.val) {
            root.right = deleteNode(root.right, key);
        } else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            TreeNode successor = root.right;
            while (successor.left != null) { successor = successor.left; }
            root.val = successor.val;
            root.right = deleteNode(root.right, successor.val);
        }
        return root;
    }
}
```

**Time** O(h) | **Space** O(h)

---

## #473 Matchsticks to Square

**Description:** Determine if matchsticks can form a square (4 equal sides) using all sticks. **Intuition:** backtrack placing each stick into one of 4 side buckets; prune when total isn't divisible by 4 or a stick exceeds the side length.

```
class Solution {
    public boolean makesquare(int[] matchsticks) {
        int total = 0;
        for (int stick : matchsticks) { total += stick; }
        if (total % 4 != 0) return false;
        int side = total / 4;
        Arrays.sort(matchsticks);
        return backtrack(matchsticks, matchsticks.length - 1, new int[4], side);
    }
    private boolean backtrack(int[] sticks, int index, int[] sides, int target) {
        if (index < 0) {
            return sides[0] == target && sides[1] == target && sides[2] == target;
        }
        for (int i = 0; i < 4; i++) {
            if (sides[i] + sticks[index] > target) continue;
            if (i > 0 && sides[i] == sides[i - 1]) continue;   // ← VARIATION: skip equal buckets
            sides[i] += sticks[index];
            if (backtrack(sticks, index - 1, sides, target)) return true;
            sides[i] -= sticks[index];
        }
        return false;
    }
}
```

**Time** $O(4^n)$ | **Space** O(n)

---

## #488 Zuma Game

**Description:** Find the minimum balls from hand to insert to clear the board (or -1). **Intuition:** DFS each board state, trying every hand ball at every insertion point, removing groups of 3+; memoize visited states.

```
class Solution {
    public int findMinStep(String board, String hand) {
        char[] handArr = hand.toCharArray();
        Arrays.sort(handArr);
        int result = dfs(board, new String(handArr), new HashMap<>());
        return result == Integer.MAX_VALUE ? -1 : result;
    }
    private int dfs(String board, String hand, Map<String, Integer> memo) {
        if (board.isEmpty()) return 0;
        if (hand.isEmpty()) return Integer.MAX_VALUE;
        String key = board + "#" + hand;
        if (memo.containsKey(key)) return memo.get(key);
        int best = Integer.MAX_VALUE;
        for (int i = 0; i < hand.length(); i++) {
            if (i > 0 && hand.charAt(i) == hand.charAt(i - 1)) continue;
            for (int j = 0; j <= board.length(); j++) {
                String next = board.substring(0, j) + hand.charAt(i) + board.substring(j);
                next = removeGroups(next);
                String remaining = hand.substring(0, i) + hand.substring(i + 1);
                int sub = dfs(next, remaining, memo);
                if (sub != Integer.MAX_VALUE) { best = Math.min(best, sub + 1); }
            }
        }
        memo.put(key, best);
        return best;
    }
    private String removeGroups(String s) {
        int i = 0, n = s.length();
        while (i < n) {
            int j = i;
            while (j < n && s.charAt(j) == s.charAt(i)) { j++; }
            if (j - i >= 3) {
                return removeGroups(s.substring(0, i) + s.substring(j));
            }
            i = j;
        }
        return s;
    }
}
```

**Time** O(exponential) | **Space** O(states)

---

## #489 Robot Room Cleaner

**Description:** Clean an entire room with a robot that has only relative move/turn/clean APIs and no map. **Intuition:** DFS with backtracking using relative coordinates; clean, try all four directions, and reverse-move to return after each branch.

```
class Solution {
    private static final int[] dr = {-1, 0, 1, 0};
    private static final int[] dc = {0, 1, 0, -1};
    public void cleanRoom(Robot robot) {
        backtrack(robot, 0, 0, 0, new HashSet<>());
    }
    private void backtrack(Robot robot, int r, int c, int dir, Set<String> visited) {
        robot.clean();
        visited.add(r + "," + c);
        for (int i = 0; i < 4; i++) {
            int nd = (dir + i) % 4;
            int nr = r + dr[nd];
            int nc = c + dc[nd];
            if (!visited.contains(nr + "," + nc) && robot.move()) {
                backtrack(robot, nr, nc, nd, visited);
                goBack(robot);
            }
            robot.turnRight();
        }
    }
    private void goBack(Robot robot) {
        robot.turnRight();
        robot.turnRight();
        robot.move();
        robot.turnRight();
        robot.turnRight();
    }
}
```

**Time** O(cells) | **Space** O(cells)

---

## #491 Increasing Subsequences

**Description:** Return all increasing subsequences of length ≥ 2 (input may have duplicates). **Intuition:** backtrack choosing elements ≥ the last picked; within one recursion level use a set to skip duplicate starts. Cannot sort (would destroy original order).

```
class Solution {
    public List<List<Integer>> findSubsequences(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }
    private void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>> result) {
        if (path.size() >= 2) { result.add(new ArrayList<>(path)); }
        Set<Integer> used = new HashSet<>();
        for (int i = start; i < nums.length; i++) {
            if (!path.isEmpty() && nums[i] < path.get(path.size() - 1)) continue;
            if (used.contains(nums[i])) continue;   // ← VARIATION: skip dup at same level
            used.add(nums[i]);
            path.add(nums[i]);
            backtrack(nums, i + 1, path, result);
            path.remove(path.size() - 1);
        }
    }
}
```

**Time** $O(2^n \cdot n)$ | **Space** O(n)

---

## #501 Find Mode in Binary Search Tree

**Description:** Find the mode(s) (most frequent values) in a BST that may have duplicates. **Intuition:** inorder gives sorted values, so equal values are adjacent; track current run count and update the mode list when counts tie or exceed the max.

```
class Solution {
    private TreeNode prev = null;
    private int count = 0, maxCount = 0;
    private List<Integer> modes = new ArrayList<>();
    public int[] findMode(TreeNode root) {
        inorder(root);
        int[] result = new int[modes.size()];
        for (int i = 0; i < modes.size(); i++) { result[i] = modes.get(i); }
        return result;
    }
    private void inorder(TreeNode node) {
        if (node == null) return;
        inorder(node.left);
        if (prev != null && prev.val == node.val) { count++; }
        else { count = 1; }
        if (count > maxCount) { maxCount = count; modes.clear(); modes.add(node.val); }
        else if (count == maxCount) { modes.add(node.val); }
        prev = node;
        inorder(node.right);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #508 Most Frequent Subtree Sum

**Description:** Find the subtree sum(s) that occur most frequently. **Intuition:** post-order compute each subtree's sum, tally frequencies in a map, then collect the keys with the max frequency.

```
class Solution {
    private Map<Integer, Integer> freq = new HashMap<>();
    private int maxFreq = 0;
    public int[] findFrequentTreeSum(TreeNode root) {
        dfs(root);
        List<Integer> result = new ArrayList<>();
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            if (e.getValue() == maxFreq) { result.add(e.getKey()); }
        }
        int[] arr = new int[result.size()];
        for (int i = 0; i < result.size(); i++) { arr[i] = result.get(i); }
        return arr;
    }
    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int sum = node.val + dfs(node.left) + dfs(node.right);
        int f = freq.merge(sum, 1, Integer::sum);
        maxFreq = Math.max(maxFreq, f);
        return sum;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #510 Inorder Successor in BST II

**Description:** Find the inorder successor of a node given only a parent pointer (no root). **Intuition:** if a right subtree exists, the successor is its leftmost node; otherwise climb up until you come from a left child.

```
class Solution {
    public Node inorderSuccessor(Node node) {
        if (node.right != null) {
            node = node.right;
            while (node.left != null) { node = node.left; }
            return node;
        }
        while (node.parent != null && node == node.parent.right) {
            node = node.parent;
        }
        return node.parent;
    }
}
```

**Time** O(h) | **Space** O(1)

---

## #513 Find Bottom Left Tree Value

**Description:** Find the leftmost value in the last (deepest) row of a binary tree. **Intuition:** BFS scanning right-to-left so the final node dequeued is the bottom-left value.

```
class Solution {
    public int findBottomLeftValue(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        TreeNode node = root;
        while (!queue.isEmpty()) {
            node = queue.poll();
            if (node.right != null) { queue.offer(node.right); }
            if (node.left != null) { queue.offer(node.left); }
        }
        return node.val;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #515 Find Largest Value in Each Tree Row

**Description:** Return the maximum value in each row of a binary tree. **Intuition:** BFS level by level, tracking the max per level.

```
class Solution {
    public List<Integer> largestValues(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            int max = Integer.MIN_VALUE;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                max = Math.max(max, node.val);
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            result.add(max);
        }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

## #526 Beautiful Arrangement

**Description:** Count permutations of 1..n where each number divides or is divisible by its position. **Intuition:** backtrack placing numbers position by position, only choosing a number when it satisfies the divisibility rule.

```
class Solution {
    private int count = 0;
    public int countArrangement(int n) {
        backtrack(n, 1, new boolean[n + 1]);
        return count;
    }
    private void backtrack(int n, int pos, boolean[] used) {
        if (pos > n) { count++; return; }
        for (int i = 1; i <= n; i++) {
            if (!used[i] && (i % pos == 0 || pos % i == 0)) {
                used[i] = true;
                backtrack(n, pos + 1, used);
                used[i] = false;
            }
        }
    }
}
```

**Time** O(k) where k = valid arrangements | **Space** O(n)

## #536 Construct Binary Tree from String

**Description:** Build a binary tree from a string like `4(2(3)(1))(6(5))`. **Intuition:** parse the leading number as the node, then the first parenthesized group as the left subtree and the second as the right subtree.

```
class Solution {
    private int index = 0;
    public TreeNode str2tree(String s) {
        if (s.isEmpty()) return null;
        return build(s);
    }
    private TreeNode build(String s) {
        int start = index;
        if (s.charAt(index) == '-') { index++; }
        while (index < s.length() && Character.isDigit(s.charAt(index))) { index++; }
        TreeNode node = new TreeNode(Integer.parseInt(s.substring(start, index)));
        if (index < s.length() && s.charAt(index) == '(') {
            index++;                        // consume '('
            node.left = build(s);
            index++;                        // consume ')'
        }
        if (index < s.length() && s.charAt(index) == '(') {
            index++;
            node.right = build(s);
            index++;
        }
        return node;
    }
}
```

**Time** O(n) | **Space** O(h)

## #538 Convert BST to Greater Tree

**Description:** Replace each node's value with the sum of all values greater than or equal to it. **Intuition:** reverse inorder (right → node → left) visits values in descending order; carry a running sum and add it into each node.

```
class Solution {
    private int sum = 0;
    public TreeNode convertBST(TreeNode root) {
        if (root == null) return null;
        convertBST(root.right);
        sum += root.val;
        root.val = sum;
        convertBST(root.left);
        return root;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #545 Boundary of Binary Tree

**Description:** Return the boundary: root, left boundary top-down, leaves left-to-right, right boundary bottom-up, no duplicates. **Intuition:** collect three parts separately — left boundary (excluding leaves), all leaves, and right boundary reversed (excluding leaves).

```
class Solution {
    public List<Integer> boundaryOfBinaryTree(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;
        if (!isLeaf(root)) { result.add(root.val); }
        addLeftBoundary(root.left, result);
        addLeaves(root, result);
        addRightBoundary(root.right, result);
        return result;
    }
    private boolean isLeaf(TreeNode node) {
        return node.left == null && node.right == null;
    }
    private void addLeftBoundary(TreeNode node, List<Integer> result) {
        while (node != null) {
            if (!isLeaf(node)) { result.add(node.val); }
            node = node.left != null ? node.left : node.right;
        }
    }
    private void addLeaves(TreeNode node, List<Integer> result) {
        if (node == null) return;
        if (isLeaf(node)) { result.add(node.val); return; }
        addLeaves(node.left, result);
        addLeaves(node.right, result);
    }
    private void addRightBoundary(TreeNode node, List<Integer> result) {
        List<Integer> temp = new ArrayList<>();
        while (node != null) {
            if (!isLeaf(node)) { temp.add(node.val); }
            node = node.right != null ? node.right : node.left;
        }
        Collections.reverse(temp);
        result.addAll(temp);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #559 Maximum Depth of N-ary Tree

**Description:** Find the maximum depth of an N-ary tree. **Intuition:** depth is 1 plus the max depth among all children.

```
class Solution {
    public int maxDepth(Node root) {
        if (root == null) return 0;
        int max = 0;
        for (Node child : root.children) {
            max = Math.max(max, maxDepth(child));
        }
        return max + 1;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #563 Binary Tree Tilt

**Description:** Sum the tilt of every node, where tilt = |left subtree sum − right subtree sum|. **Intuition:** post-order return each subtree sum; accumulate the absolute difference into a global total.

```
class Solution {
    private int total = 0;
    public int findTilt(TreeNode root) {
        sum(root);
        return total;
    }
    private int sum(TreeNode node) {
        if (node == null) return 0;
        int left = sum(node.left);
        int right = sum(node.right);
        total += Math.abs(left - right);
        return left + right + node.val;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #572 Subtree of Another Tree

**Description:** Check whether `subRoot` is a subtree of `root`. **Intuition:** at every node of `root`, test for structural equality with `subRoot`.

```
class Solution {
    public boolean isSubtree(TreeNode root, TreeNode subRoot) {
        if (root == null) return false;
        if (isSame(root, subRoot)) return true;
        return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
    }
    private boolean isSame(TreeNode a, TreeNode b) {
        if (a == null && b == null) return true;
        if (a == null || b == null) return false;
        if (a.val != b.val) return false;
        return isSame(a.left, b.left) && isSame(a.right, b.right);
    }
}
```

**Time** O(m·n) | **Space** O(h)

---

## #589 N-ary Tree Preorder Traversal

**Description:** Return the preorder traversal of an N-ary tree. **Intuition:** visit the node, then recurse into each child in order.

```
class Solution {
    public List<Integer> preorder(Node root) {
        List<Integer> result = new ArrayList<>();
        dfs(root, result);
        return result;
    }
    private void dfs(Node node, List<Integer> result) {
        if (node == null) return;
        result.add(node.val);
        for (Node child : node.children) { dfs(child, result); }
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #590 N-ary Tree Postorder Traversal

**Description:** Return the postorder traversal of an N-ary tree. **Intuition:** recurse into all children first, then visit the node.

```
class Solution {
    public List<Integer> postorder(Node root) {
        List<Integer> result = new ArrayList<>();
        dfs(root, result);
        return result;
    }
    private void dfs(Node node, List<Integer> result) {
        if (node == null) return;
        for (Node child : node.children) { dfs(child, result); }
        result.add(node.val);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #606 Construct String from Binary Tree

**Description:** Build a preorder string with parentheses, e.g. `1(2(4))(3)`. **Intuition:** append the value, then each non-null child wrapped in parentheses; keep empty `()` for a left child only when a right child exists.

```
class Solution {
    public String tree2str(TreeNode root) {
        StringBuilder builder = new StringBuilder();
        dfs(root, builder);
        return builder.toString();
    }
    private void dfs(TreeNode node, StringBuilder builder) {
        if (node == null) return;
        builder.append(node.val);
        if (node.left == null && node.right == null) return;
        builder.append('(');
        dfs(node.left, builder);
        builder.append(')');
        if (node.right != null) {
            builder.append('(');
            dfs(node.right, builder);
            builder.append(')');
        }
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #617 Merge Two Binary Trees

**Description:** Merge two binary trees by summing overlapping node values. **Intuition:** if either node is null, return the other; otherwise sum values and recurse on both child pairs.

```
class Solution {
    public TreeNode mergeTrees(TreeNode root1, TreeNode root2) {
        if (root1 == null) return root2;
        if (root2 == null) return root1;
        TreeNode merged = new TreeNode(root1.val + root2.val);
        merged.left = mergeTrees(root1.left, root2.left);
        merged.right = mergeTrees(root1.right, root2.right);
        return merged;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #637 Average of Levels in Binary Tree

**Description:** Return the average value of nodes on each level. **Intuition:** BFS each level, summing values and dividing by the level size.

```
class Solution {
    public List<Double> averageOfLevels(TreeNode root) {
        List<Double> result = new ArrayList<>();
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            double sum = 0;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                sum += node.val;
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            result.add(sum / size);
        }
        return result;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #652 Find Duplicate Subtrees

**Description:** Return one root per group of structurally identical, duplicated subtrees. **Intuition:** serialize each subtree post-order into a canonical string; the second time a serialization appears, record its root.

```
class Solution {
    public List<TreeNode> findDuplicateSubtrees(TreeNode root) {
        List<TreeNode> result = new ArrayList<>();
        serialize(root, new HashMap<>(), result);
        return result;
    }
    private String serialize(TreeNode node, Map<String, Integer> seen, List<TreeNode> result) {
        if (node == null) return "#";
        String key = node.val + "," + serialize(node.left, seen, result) + "," + serialize(node.right, seen, result);
        int count = seen.merge(key, 1, Integer::sum);
        if (count == 2) { result.add(node); }
        return key;
    }
}
```

**Time** O($n^2$) | **Space** O($n^2$)

---

## #653 Two Sum IV - Input is a BST

**Description:** Find whether two nodes in a BST sum to a target. **Intuition:** traverse, storing seen values in a set; for each node check if `target - val` was seen.

```
class Solution {
    public boolean findTarget(TreeNode root, int k) {
        return dfs(root, k, new HashSet<>());
    }
    private boolean dfs(TreeNode node, int k, Set<Integer> seen) {
        if (node == null) return false;
        if (seen.contains(k - node.val)) return true;
        seen.add(node.val);
        return dfs(node.left, k, seen) || dfs(node.right, k, seen);
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #654 Maximum Binary Tree

**Description:** Build a tree where the root is the array max, left subtree from the left subarray, right subtree from the right subarray. **Intuition:** find the max in the range as root, recurse on the two halves.

```
class Solution {
    public TreeNode constructMaximumBinaryTree(int[] nums) {
        return build(nums, 0, nums.length - 1);
    }
    private TreeNode build(int[] nums, int lo, int hi) {
        if (lo > hi) return null;
        int maxIndex = lo;
        for (int i = lo + 1; i <= hi; i++) {
            if (nums[i] > nums[maxIndex]) { maxIndex = i; }
        }
        TreeNode root = new TreeNode(nums[maxIndex]);
        root.left = build(nums, lo, maxIndex - 1);
        root.right = build(nums, maxIndex + 1, hi);
        return root;
    }
}
```

**Time** O($n^2$) | **Space** O(n)

---

## #662 Maximum Width of Binary Tree

**Description:** Return the maximum width of any level, where width counts the gap between the leftmost and rightmost non-null nodes. **Intuition:** assign heap-style indices (left = 2i, right = 2i+1); per level the width is last index − first index + 1.

```
class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        if (root == null) return 0;
        int max = 0;
        Queue<TreeNode> queue = new ArrayDeque<>();
        Queue<Integer> indices = new ArrayDeque<>();
        queue.offer(root);
        indices.offer(0);
        while (!queue.isEmpty()) {
            int size = queue.size();
            int first = 0, last = 0;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                int index = indices.poll();
                if (i == 0) { first = index; }
                if (i == size - 1) { last = index; }
                if (node.left != null) { queue.offer(node.left); indices.offer(2 * index); }
                if (node.right != null) { queue.offer(node.right); indices.offer(2 * index + 1); }
            }
            max = Math.max(max, last - first + 1);
        }
        return max;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #671 Second Minimum Node In a Binary Tree

**Description:** In a tree where each node's value is the min of its children, find the second smallest value overall. **Intuition:** the root is the global minimum; DFS for the smallest value strictly greater than the root.

```
class Solution {
    public int findSecondMinimumValue(TreeNode root) {
        long second = dfs(root, root.val);
        return second == Long.MAX_VALUE ? -1 : (int) second;
    }
    private long dfs(TreeNode node, int min) {
        if (node == null) return Long.MAX_VALUE;
        if (node.val > min) return node.val;
        long left = dfs(node.left, min);
        long right = dfs(node.right, min);
        return Math.min(left, right);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #679 24 Game

**Description:** Determine if four numbers can be combined with +, −, ×, ÷ to make 24. **Intuition:** backtrack: pick any two numbers, apply each operator, replace them with the result, and recurse until one number remains.

```
class Solution {
    public boolean judgePoint24(int[] cards) {
        List<Double> list = new ArrayList<>();
        for (int card : cards) { list.add((double) card); }
        return backtrack(list);
    }
    private boolean backtrack(List<Double> nums) {
        if (nums.size() == 1) { return Math.abs(nums.get(0) - 24) < 1e-6; }
        for (int i = 0; i < nums.size(); i++) {
            for (int j = 0; j < nums.size(); j++) {
                if (i == j) continue;
                List<Double> next = new ArrayList<>();
                for (int k = 0; k < nums.size(); k++) {
                    if (k != i && k != j) { next.add(nums.get(k)); }
                }
                for (double value : compute(nums.get(i), nums.get(j))) {
                    next.add(value);
                    if (backtrack(next)) return true;
                    next.remove(next.size() - 1);
                }
            }
        }
        return false;
    }
    private List<Double> compute(double a, double b) {
        List<Double> result = new ArrayList<>(Arrays.asList(a + b, a - b, a * b));
        if (Math.abs(b) > 1e-6) { result.add(a / b); }
        return result;
    }
}
```

**Time** O(1) (bounded) | **Space** O(1)

---

## #698 Partition to K Equal Sum Subsets

**Description:** Determine if an array can be split into k subsets of equal sum. **Intuition:** target = total/k; backtrack filling one bucket at a time, sorting descending to fail fast, skipping when total isn't divisible.

```
class Solution {
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int total = 0;
        for (int num : nums) { total += num; }
        if (total % k != 0) return false;
        int target = total / k;
        Arrays.sort(nums);
        if (nums[nums.length - 1] > target) return false;
        return backtrack(nums, nums.length - 1, new boolean[nums.length], k, 0, target);
    }
    private boolean backtrack(int[] nums, int index, boolean[] used, int k, int current, int target) {
        if (k == 0) return true;
        if (current == target) {
            return backtrack(nums, nums.length - 1, used, k - 1, 0, target);
        }
        for (int i = index; i >= 0; i--) {
            if (used[i] || current + nums[i] > target) continue;
            used[i] = true;
            if (backtrack(nums, i - 1, used, k, current + nums[i], target)) return true;
            used[i] = false;
        }
        return false;
    }
}
```

**Time** $O(k \cdot 2^n)$ | **Space** O(n)

## #700 Search in a Binary Search Tree

**Description:** Find the subtree rooted at the node with a given value in a BST. **Intuition:** binary-search the tree: go left if target is smaller, right if larger.

```
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        while (root != null && root.val != val) {
            root = val < root.val ? root.left : root.right;
        }
        return root;
    }
}
```

**Time** O(h) | **Space** O(1)

## #701 Insert into a Binary Search Tree

**Description:** Insert a value into a BST and return the root. **Intuition:** descend like a search until reaching a null child, then attach a new node there.

```
class Solution {
    public TreeNode insertIntoBST(TreeNode root, int val) {
        if (root == null) return new TreeNode(val);
        if (val < root.val) { root.left = insertIntoBST(root.left, val); }
        else { root.right = insertIntoBST(root.right, val); }
        return root;
    }
}
```

**Time** O(h) | **Space** O(h)

## #742 Closest Leaf in a Binary Tree

**Description:** Find the value of the leaf nearest to the node with a given value. **Intuition:** convert the tree to an undirected graph, then BFS from the target node until the first leaf is reached.

```
class Solution {
    public int findClosestLeaf(TreeNode root, int k) {
        Map<TreeNode, List<TreeNode>> graph = new HashMap<>();
        TreeNode[] start = new TreeNode[1];
        buildGraph(root, null, graph, k, start);
        Queue<TreeNode> queue = new ArrayDeque<>();
        Set<TreeNode> visited = new HashSet<>();
        queue.offer(start[0]);
        visited.add(start[0]);
        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            if (node.left == null && node.right == null) { return node.val; }
            for (TreeNode neighbor : graph.getOrDefault(node, new ArrayList<>())) {
                if (visited.add(neighbor)) { queue.offer(neighbor); }
            }
        }
        return -1;
    }
    private void buildGraph(TreeNode node, TreeNode parent, Map<TreeNode, List<TreeNode>> graph, int k, TreeNode[] start) {
        if (node == null) return;
        if (node.val == k) { start[0] = node; }
        if (parent != null) {
            graph.computeIfAbsent(node, x -> new ArrayList<>()).add(parent);
            graph.computeIfAbsent(parent, x -> new ArrayList<>()).add(node);
        }
        buildGraph(node.left, node, graph, k, start);
        buildGraph(node.right, node, graph, k, start);
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #776 Split BST

**Description:** Split a BST into two trees: one with values ≤ V and one with values > V. **Intuition:** recurse; at each node decide which result tree it belongs to based on V, then reattach the recursively-split child.

```
class Solution {
    public TreeNode[] splitBST(TreeNode root, int target) {
        if (root == null) return new TreeNode[]{null, null};
        if (root.val <= target) {
            TreeNode[] right = splitBST(root.right, target);
            root.right = right[0];
            return new TreeNode[]{root, right[1]};
        } else {
            TreeNode[] left = splitBST(root.left, target);
            root.left = left[1];
            return new TreeNode[]{left[0], root};
        }
    }
}
```

**Time** O(h) | **Space** O(h)

---

## #784 Letter Case Permutation

**Description:** Return all strings formable by toggling the case of letters in a string. **Intuition:** backtrack character by character; digits have one choice, letters branch into lower and upper case.

```java
class Solution {
    public List<String> letterCasePermutation(String s) {
        List<String> result = new ArrayList<>();
        backtrack(s.toCharArray(), 0, result);
        return result;
    }
    private void backtrack(char[] arr, int start, List<String> result) {
        if (start == arr.length) { result.add(new String(arr)); return; }
        if (Character.isLetter(arr[start])) {
            arr[start] = Character.toLowerCase(arr[start]);
            backtrack(arr, start + 1, result);
            arr[start] = Character.toUpperCase(arr[start]);
            backtrack(arr, start + 1, result);
        } else {
            backtrack(arr, start + 1, result);
        }
    }
}
```

**Time** $O(2^n \cdot n)$ | **Space** $O(n)$

---

## #814 Binary Tree Pruning

**Description:** Remove every subtree that contains no node with value 1. **Intuition:** post-order prune children first; drop a node if both children become null and its own value is 0.

```java
class Solution {
    public TreeNode pruneTree(TreeNode root) {
        if (root == null) return null;
        root.left = pruneTree(root.left);
        root.right = pruneTree(root.right);
        if (root.left == null && root.right == null && root.val == 0) {
            return null;
        }
        return root;
    }
}
```

**Time** $O(n)$ | **Space** $O(h)$

---

## #842 Split Array into Fibonacci Sequence

**Description:** Split a digit string into a Fibonacci-like sequence (each term = sum of previous two), with terms fitting in a 32-bit int. **Intuition:** backtrack the first two numbers; the rest is forced. Prune leading zeros and overflow.

```
class Solution {
    public List<Integer> splitIntoFibonacci(String num) {
        List<Integer> result = new ArrayList<>();
        backtrack(num, 0, result);
        return result;
    }
    private boolean backtrack(String num, int start, List<Integer> path) {
        if (start == num.length()) { return path.size() >= 3; }
        for (int i = start; i < num.length(); i++) {
            if (num.charAt(start) == '0' && i > start) break;   // no leading zero
            long value = Long.parseLong(num.substring(start, i + 1));
            if (value > Integer.MAX_VALUE) break;
            int size = path.size();
            if (size >= 2 && value > (long) path.get(size - 1) + path.get(size - 2)) break;
            if (size < 2 || value == (long) path.get(size - 1) + path.get(size - 2)) {
                path.add((int) value);
                if (backtrack(num, i + 1, path)) return true;
                path.remove(path.size() - 1);
            }
        }
        return false;
    }
}
```

**Time** O(n²) | **Space** O(n)

---

## #863 All Nodes Distance K in Binary Tree

**Description:** Find all node values exactly distance k from a target node. **Intuition:** build parent links so the tree becomes an undirected graph, then BFS k levels from the target.

```
class Solution {
    public List<Integer> distanceK(TreeNode root, TreeNode target, int k) {
        Map<TreeNode, TreeNode> parent = new HashMap<>();
        buildParents(root, null, parent);
        Queue<TreeNode> queue = new ArrayDeque<>();
        Set<TreeNode> visited = new HashSet<>();
        queue.offer(target);
        visited.add(target);
        int dist = 0;
        while (!queue.isEmpty()) {
            if (dist == k) break;
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                for (TreeNode neighbor : new TreeNode[]{node.left, node.right, parent.get(node)}) {
                    if (neighbor != null && visited.add(neighbor)) { queue.offer(neighbor); }
                }
            }
            dist++;
        }
        List<Integer> result = new ArrayList<>();
        for (TreeNode node : queue) { result.add(node.val); }
        return result;
    }
    private void buildParents(TreeNode node, TreeNode par, Map<TreeNode, TreeNode> parent) {
        if (node == null) return;
        parent.put(node, par);
        buildParents(node.left, node, parent);
        buildParents(node.right, node, parent);
    }
}
```

**Time** O(n) | **Space** O(n)

## #865 Smallest Subtree with all the Deepest Nodes

**Description:** Find the smallest subtree containing all of the tree's deepest leaves. **Intuition:** post-order return (depth, lca). If left and right depths tie, the current node is the LCA; otherwise carry up the deeper side.

```
class Solution {
    public TreeNode subtreeWithAllDeepest(TreeNode root) {
        return dfs(root).node;
    }
    private Result dfs(TreeNode node) {
        if (node == null) return new Result(0, null);
        Result left = dfs(node.left);
        Result right = dfs(node.right);
        if (left.depth == right.depth) {
            return new Result(left.depth + 1, node);   // ← VARIATION: tie → this node is LCA
        }
        return left.depth > right.depth
            ? new Result(left.depth + 1, left.node)
            : new Result(right.depth + 1, right.node);
    }
    private static class Result {
        int depth;
        TreeNode node;
        Result(int depth, TreeNode node) { this.depth = depth; this.node = node; }
    }
}
```

**Time** O(n) | **Space** O(h)

## #889 Construct Binary Tree from Preorder and Postorder Traversal

**Description:** Reconstruct a binary tree from preorder and postorder traversals (any valid tree). **Intuition:** preorder[start] is the root; preorder[start+1] is the left subtree root, whose position in postorder gives the left subtree size.

```
class Solution {
    private int preIndex = 0, postIndex = 0;
    public TreeNode constructFromPrePost(int[] preorder, int[] postorder) {
        TreeNode root = new TreeNode(preorder[preIndex++]);
        if (root.val != postorder[postIndex]) {
            root.left = constructFromPrePost(preorder, postorder);
        }
        if (root.val != postorder[postIndex]) {
            root.right = constructFromPrePost(preorder, postorder);
        }
        postIndex++;
        return root;
    }
}
```

**Time** O(n) | **Space** O(h)

## #897 Increasing Order Search Tree

**Description:** Rearrange a BST into a right-skewed tree following inorder order. **Intuition:** inorder traversal; relink each node as the right child of the previous, clearing left pointers.

```
class Solution {
    private TreeNode current;
    public TreeNode increasingBST(TreeNode root) {
        TreeNode dummy = new TreeNode(0);
        current = dummy;
        inorder(root);
        return dummy.right;
    }
    private void inorder(TreeNode node) {
        if (node == null) return;
        inorder(node.left);
        node.left = null;
        current.right = node;
        current = node;
        inorder(node.right);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #919 Complete Binary Tree Inserter

**Description:** Design a structure that supports O(1) insertion into a complete binary tree. **Intuition:** keep a BFS queue of nodes that still have a free child slot; insert into the front node and enqueue the new node.

```
class CBTInserter {
    private TreeNode root;
    private Queue<TreeNode> queue = new ArrayDeque<>();
    public CBTInserter(TreeNode root) {
        this.root = root;
        Queue<TreeNode> bfs = new ArrayDeque<>();
        bfs.offer(root);
        while (!bfs.isEmpty()) {
            TreeNode node = bfs.poll();
            if (node.left != null) { bfs.offer(node.left); }
            if (node.right != null) { bfs.offer(node.right); }
            if (node.left == null || node.right == null) { queue.offer(node); }
        }
    }
    public int insert(int val) {
        TreeNode node = new TreeNode(val);
        TreeNode parent = queue.peek();
        if (parent.left == null) { parent.left = node; }
        else { parent.right = node; queue.poll(); }
        queue.offer(node);
        return parent.val;
    }
    public TreeNode get_root() { return root; }
}
```

**Time** O(1) insert | **Space** O(n)

---

## #932 Beautiful Array

**Description:** Construct an array of 1..n where no three indices i<j<k satisfy 2·a[j] = a[i] + a[k]. **Intuition:** divide and conquer: a beautiful array of odds followed by evens stays beautiful, since odd + even is never twice an integer.

```
class Solution {
    public int[] beautifulArray(int n) {
        List<Integer> result = new ArrayList<>();
        result.add(1);
        while (result.size() < n) {
            List<Integer> next = new ArrayList<>();
            for (int x : result) { if (2 * x - 1 <= n) { next.add(2 * x - 1); } }   // odds
            for (int x : result) { if (2 * x <= n) { next.add(2 * x); } }           // evens
            result = next;
        }
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) { arr[i] = result.get(i); }
        return arr;
    }
}
```

**Time** O(n log n) | **Space** O(n)

---

## #938 Range Sum of BST

**Description:** Sum all node values within the inclusive range [low, high]. **Intuition:** prune branches outside the range using BST ordering.

```
class Solution {
    public int rangeSumBST(TreeNode root, int low, int high) {
        if (root == null) return 0;
        if (root.val < low) return rangeSumBST(root.right, low, high);
        if (root.val > high) return rangeSumBST(root.left, low, high);
        return root.val + rangeSumBST(root.left, low, high) + rangeSumBST(root.right, low, high);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #951 Flip Equivalent Binary Trees

**Description:** Check if two trees are flip-equivalent (can be made identical by swapping some children). **Intuition:** match children either directly or swapped at each node.

```
class Solution {
    public boolean flipEquiv(TreeNode root1, TreeNode root2) {
        if (root1 == null && root2 == null) return true;
        if (root1 == null || root2 == null) return false;
        if (root1.val != root2.val) return false;
        return (flipEquiv(root1.left, root2.left) && flipEquiv(root1.right, root2.right))
            || (flipEquiv(root1.left, root2.right) && flipEquiv(root1.right, root2.left));
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #958 Check Completeness of a Binary Tree

**Description:** Determine if a binary tree is complete. **Intuition:** BFS including null children; once a null is seen, no real node may follow.

```
class Solution {
    public boolean isCompleteTree(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        boolean seenNull = false;
        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            if (node == null) {
                seenNull = true;
            } else {
                if (seenNull) return false;   // ← VARIATION: real node after a gap
                queue.offer(node.left);
                queue.offer(node.right);
            }
        }
        return true;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #987 Vertical Order Traversal of a Binary Tree

**Description:** Group node values by column, then by row, then by value within the same position. **Intuition:** DFS recording (col, row, val); sort by column, then row, then value.

```
class Solution {
    public List<List<Integer>> verticalTraversal(TreeNode root) {
        List<int[]> nodes = new ArrayList<>();
        dfs(root, 0, 0, nodes);
        nodes.sort((a, b) -> {
            if (a[0] != b[0]) return Integer.compare(a[0], b[0]);
            if (a[1] != b[1]) return Integer.compare(a[1], b[1]);
            return Integer.compare(a[2], b[2]);
        });
        List<List<Integer>> result = new ArrayList<>();
        Integer prevCol = null;
        for (int[] node : nodes) {
            if (prevCol == null || node[0] != prevCol) {
                result.add(new ArrayList<>());
                prevCol = node[0];
            }
            result.get(result.size() - 1).add(node[2]);
        }
        return result;
    }
    private void dfs(TreeNode node, int row, int col, List<int[]> nodes) {
        if (node == null) return;
        nodes.add(new int[]{col, row, node.val});
        dfs(node.left, row + 1, col - 1, nodes);
        dfs(node.right, row + 1, col + 1, nodes);
    }
}
```

**Time** O(n log n) | **Space** O(n)

---

## #988 Smallest String Starting From Leaf

**Description:** Return the lexicographically smallest root-to-leaf string, where each node maps to a letter 'a'..'z' and the string is read leaf to root. **Intuition:** build the path down, reverse it at each leaf, and track the minimum.

```
class Solution {
    private String smallest = null;
    public String smallestFromLeaf(TreeNode root) {
        dfs(root, new StringBuilder());
        return smallest;
    }
    private void dfs(TreeNode node, StringBuilder builder) {
        if (node == null) return;
        builder.append((char) ('a' + node.val));
        if (node.left == null && node.right == null) {
            String candidate = builder.reverse().toString();
            builder.reverse();
            if (smallest == null || candidate.compareTo(smallest) < 0) { smallest = candidate; }
        }
        dfs(node.left, builder);
        dfs(node.right, builder);
        builder.deleteCharAt(builder.length() - 1);
    }
}
```

**Time** O(n·h) | **Space** O(h)

---

## #993 Cousins in Binary Tree

**Description:** Check if two values are cousins (same depth, different parents). **Intuition:** BFS recording each value's depth and parent; cousins share depth but differ in parent.

```
class Solution {
    public boolean isCousins(TreeNode root, int x, int y) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int size = queue.size();
            boolean foundX = false, foundY = false;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (node.val == x) { foundX = true; }
                if (node.val == y) { foundY = true; }
                if (node.left != null && node.right != null) {
                    int a = node.left.val, b = node.right.val;
                    if ((a == x && b == y) || (a == y && b == x)) return false;   // siblings
                }
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            if (foundX && foundY) return true;
            if (foundX || foundY) return false;
        }
        return false;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #998 Maximum Binary Tree II

**Description:** Insert a value into a maximum tree built by appending the value to the original array's end. **Intuition:** since the value was appended last, it lies on the rightmost spine; insert where it first exceeds the current node.

```
class Solution {
    public TreeNode insertIntoMaxTree(TreeNode root, int val) {
        if (root == null || val > root.val) {
            TreeNode node = new TreeNode(val);
            node.left = root;
            return node;
        }
        root.right = insertIntoMaxTree(root.right, val);
        return root;
    }
}
```

**Time** O(h) | **Space** O(h)

---

## #1008 Construct Binary Search Tree from Preorder Traversal

**Description:** Build a BST from its preorder traversal. **Intuition:** the first value is the root; values less than it form the left subtree, the rest form the right. Use an upper bound to decide where to stop.

```
class Solution {
    private int index = 0;
    public TreeNode bstFromPreorder(int[] preorder) {
        return build(preorder, Integer.MAX_VALUE);
    }
    private TreeNode build(int[] preorder, int bound) {
        if (index == preorder.length || preorder[index] > bound) return null;
        TreeNode root = new TreeNode(preorder[index++]);
        root.left = build(preorder, root.val);
        root.right = build(preorder, bound);
        return root;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #1026 Maximum Difference Between Node and Ancestor

**Description:** Find the maximum |a – d| over all ancestor a and descendant d pairs. **Intuition:** pass the min and max seen on the path down; at each node update the answer with the largest gap against those extremes.

```
class Solution {
    private int max = 0;
    public int maxAncestorDiff(TreeNode root) {
        dfs(root, root.val, root.val);
        return max;
    }
    private void dfs(TreeNode node, int min, int maxVal) {
        if (node == null) return;
        max = Math.max(max, Math.max(Math.abs(node.val - min), Math.abs(node.val - maxVal)));
        min = Math.min(min, node.val);
        maxVal = Math.max(maxVal, node.val);
        dfs(node.left, min, maxVal);
        dfs(node.right, min, maxVal);
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #1038 Binary Search Tree to Greater Sum Tree

**Description:** Replace each node's value with the sum of all values greater than or equal to it. **Intuition:** reverse inorder (right → node → left) processes descending order; carry a running sum.

```
class Solution {
    private int sum = 0;
    public TreeNode bstToGst(TreeNode root) {
        if (root == null) return null;
        bstToGst(root.right);
        sum += root.val;
        root.val = sum;
        bstToGst(root.left);
        return root;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #1087 Brace Expansion

**Description:** Expand a string with brace options like `{a,b}c{d,e}` into all sorted concatenations. **Intuition:** parse each position into a sorted list of choices, then backtrack the Cartesian product.

```
class Solution {
    public String[] expand(String s) {
        List<List<Character>> groups = new ArrayList<>();
        int i = 0;
        while (i < s.length()) {
            List<Character> options = new ArrayList<>();
            if (s.charAt(i) == '{') {
                i++;
                while (s.charAt(i) != '}') {
                    if (s.charAt(i) != ',') { options.add(s.charAt(i)); }
                    i++;
                }
                i++;   // skip '}'
            } else {
                options.add(s.charAt(i));
                i++;
            }
            Collections.sort(options);
            groups.add(options);
        }
        List<String> result = new ArrayList<>();
        backtrack(groups, 0, new StringBuilder(), result);
        return result.toArray(new String[0]);
    }
    private void backtrack(List<List<Character>> groups, int start, StringBuilder builder, List<String> result) {
        if (start == groups.size()) { result.add(builder.toString()); return; }
        for (char c : groups.get(start)) {
            builder.append(c);
            backtrack(groups, start + 1, builder, result);
            builder.deleteCharAt(builder.length() - 1);
        }
    }
}
```

**Time** O(product of group sizes) | **Space** O(n)

---

## #1104 Path In Zigzag Labelled Binary Tree

**Description:** Return the root-to-node path of labels in a zigzag-labeled infinite binary tree. **Intuition:** the normal parent of label is label/2, but rows alternate direction, so mirror the parent within its row range.

```
class Solution {
    public List<Integer> pathInZigZagTree(int label) {
        int level = 0;
        while ((1 << (level + 1)) <= label) { level++; }
        LinkedList<Integer> result = new LinkedList<>();
        while (label >= 1) {
            result.addFirst(label);
            int levelMax = (1 << (level + 1)) - 1;
            int levelMin = 1 << level;
            label = (levelMin + levelMax - label) / 2;   // ← VARIATION: mirror then halve
            level--;
        }
        return result;
    }
}
```

**Time** O(log n) | **Space** O(log n)

---

## #1110 Delete Nodes And Return Forest

**Description:** Delete the given values and return the roots of the resulting forest. **Intuition:** post-order; if a node is deleted, its surviving children become new roots. A node is a root if its parent was deleted (or it's the original root) and it itself survives.

```
class Solution {
    public List<TreeNode> delNodes(TreeNode root, int[] to_delete) {
        Set<Integer> toDelete = new HashSet<>();
        for (int value : to_delete) { toDelete.add(value); }
        List<TreeNode> result = new ArrayList<>();
        if (dfs(root, true, toDelete, result) != null) { result.add(root); }
        return result;
    }
    private TreeNode dfs(TreeNode node, boolean isRoot, Set<Integer> toDelete, List<TreeNode> result) {
        if (node == null) return null;
        boolean deleted = toDelete.contains(node.val);
        if (isRoot && !deleted) { result.add(node); }
        node.left = dfs(node.left, deleted, toDelete, result);
        node.right = dfs(node.right, deleted, toDelete, result);
        return deleted ? null : node;
    }
}
```

**Time** O(n) | **Space** O(h)

Note: the helper handles adding roots; the public method's extra add is avoided by passing `isRoot` — here roots are added inside `dfs`, so the call simply discards its return.

---

## #1123 Lowest Common Ancestor of Deepest Leaves

**Description:** Find the LCA of all the deepest leaves. **Intuition:** post-order return (depth, lca); when both arms reach equal depth, the node is the LCA, else carry up the deeper arm.

```
class Solution {
    public TreeNode lcaDeepestLeaves(TreeNode root) {
        return dfs(root).node;
    }
    private Result dfs(TreeNode node) {
        if (node == null) return new Result(0, null);
        Result left = dfs(node.left);
        Result right = dfs(node.right);
        if (left.depth == right.depth) {
            return new Result(left.depth + 1, node);
        }
        return left.depth > right.depth
            ? new Result(left.depth + 1, left.node)
            : new Result(right.depth + 1, right.node);
    }
    private static class Result {
        int depth;
        TreeNode node;
        Result(int depth, TreeNode node) { this.depth = depth; this.node = node; }
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #1161 Maximum Level Sum of a Binary Tree

**Description:** Return the 1-indexed level with the largest sum of node values. **Intuition:** BFS level by level, tracking the level whose sum is maximum.

```
class Solution {
    public int maxLevelSum(TreeNode root) {
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        int level = 0, bestLevel = 1, maxSum = Integer.MIN_VALUE;
        while (!queue.isEmpty()) {
            level++;
            int size = queue.size();
            int sum = 0;
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                sum += node.val;
                if (node.left != null) { queue.offer(node.left); }
                if (node.right != null) { queue.offer(node.right); }
            }
            if (sum > maxSum) { maxSum = sum; bestLevel = level; }
        }
        return bestLevel;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #1214 Two Sum BSTs

**Description:** Given two BSTs and a target, decide if a value from each sums to the target. **Intuition:** store all values of the first BST in a set, then traverse the second checking for the complement.

```
class Solution {
    public boolean twoSumBSTs(TreeNode root1, TreeNode root2, int target) {
        Set<Integer> seen = new HashSet<>();
        collect(root1, seen);
        return search(root2, target, seen);
    }
    private void collect(TreeNode node, Set<Integer> seen) {
        if (node == null) return;
        seen.add(node.val);
        collect(node.left, seen);
        collect(node.right, seen);
    }
    private boolean search(TreeNode node, int target, Set<Integer> seen) {
        if (node == null) return false;
        if (seen.contains(target - node.val)) return true;
        return search(node.left, target, seen) || search(node.right, target, seen);
    }
}
```

**Time** O(m + n) | **Space** O(m)

---

## #1305 All Elements in Two Binary Search Trees

**Description:** Return all elements from two BSTs in sorted order. **Intuition:** inorder each tree into a sorted list, then merge the two lists like merge sort.

```
class Solution {
    public List<Integer> getAllElements(TreeNode root1, TreeNode root2) {
        List<Integer> a = new ArrayList<>(), b = new ArrayList<>();
        inorder(root1, a);
        inorder(root2, b);
        List<Integer> result = new ArrayList<>();
        int i = 0, j = 0;
        while (i < a.size() && j < b.size()) {
            if (a.get(i) <= b.get(j)) { result.add(a.get(i++)); }
            else { result.add(b.get(j++)); }
        }
        while (i < a.size()) { result.add(a.get(i++)); }
        while (j < b.size()) { result.add(b.get(j++)); }
        return result;
    }
    private void inorder(TreeNode node, List<Integer> list) {
        if (node == null) return;
        inorder(node.left, list);
        list.add(node.val);
        inorder(node.right, list);
    }
}
```

**Time** O(m + n) | **Space** O(m + n)

---

## #1361 Validate Binary Tree Nodes

**Description:** Given child arrays, verify the nodes form exactly one valid binary tree. **Intuition:** exactly one node has no parent (the root); every other node has exactly one parent, there are no cycles, and all nodes are reachable from the root.

```
class Solution {
    public boolean validateBinaryTreeNodes(int n, int[] leftChild, int[] rightChild) {
        int[] indegree = new int[n];
        for (int i = 0; i < n; i++) {
            if (leftChild[i] != -1) { if (++indegree[leftChild[i]] > 1) return false; }
            if (rightChild[i] != -1) { if (++indegree[rightChild[i]] > 1) return false; }
        }
        int root = -1;
        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0) {
                if (root != -1) return false;   // more than one root
                root = i;
            }
        }
        if (root == -1) return false;
        int count = 0;
        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
            int node = queue.poll();
            count++;
            if (leftChild[node] != -1) { queue.offer(leftChild[node]); }
            if (rightChild[node] != -1) { queue.offer(rightChild[node]); }
        }
        return count == n;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #1382 Balance a Binary Search Tree

**Description:** Rebuild a BST so it becomes height-balanced. **Intuition:** inorder gives a sorted array; build a balanced BST by recursively choosing the middle element as root.

```
class Solution {
    public TreeNode balanceBST(TreeNode root) {
        List<Integer> values = new ArrayList<>();
        inorder(root, values);
        return build(values, 0, values.size() - 1);
    }
    private void inorder(TreeNode node, List<Integer> values) {
        if (node == null) return;
        inorder(node.left, values);
        values.add(node.val);
        inorder(node.right, values);
    }
    private TreeNode build(List<Integer> values, int lo, int hi) {
        if (lo > hi) return null;
        int mid = lo + (hi - lo) / 2;
        TreeNode root = new TreeNode(values.get(mid));
        root.left = build(values, lo, mid - 1);
        root.right = build(values, mid + 1, hi);
        return root;
    }
}
```

**Time** O(n) | **Space** O(n)

---

## #1522 Diameter of N-Ary Tree

**Description:** Find the diameter (longest path between any two nodes) of an N-ary tree. **Intuition:** like binary-tree diameter, but track the two deepest child heights to form the longest path through each node.

```
class Solution {
    private int diameter = 0;
    public int diameter(Node root) {
        dfs(root);
        return diameter;
    }
    private int dfs(Node node) {
        if (node == null) return 0;
        int max1 = 0, max2 = 0;
        for (Node child : node.children) {
            int height = dfs(child);
            if (height > max1) { max2 = max1; max1 = height; }
            else if (height > max2) { max2 = height; }
        }
        diameter = Math.max(diameter, max1 + max2);
        return max1 + 1;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #1644 Lowest Common Ancestor of a Binary Tree II

**Description:** Find the LCA of two nodes that may not both exist in the tree; return null if either is missing. **Intuition:** standard LCA but count how many of p, q were actually found, so a node returned without seeing both isn't accepted.

```
class Solution {
    private int found = 0;
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode lca = dfs(root, p, q);
        return found == 2 ? lca : null;
    }
    private TreeNode dfs(TreeNode node, TreeNode p, TreeNode q) {
        if (node == null) return null;
        TreeNode left = dfs(node.left, p, q);
        TreeNode right = dfs(node.right, p, q);
        if (node == p || node == q) {
            found++;
            return node;
        }
        if (left != null && right != null) return node;
        return left != null ? left : right;
    }
}
```

**Time** O(n) | **Space** O(h)

---

## #1650 Lowest Common Ancestor of a Binary Tree III

**Description:** Find the LCA of two nodes given parent pointers, without root access. **Intuition:** like finding the intersection of two linked lists — walk both pointers up, switching to the other node when one reaches the top; they meet at the LCA.

```
class Solution {
    public Node lowestCommonAncestor(Node p, Node q) {
        Node a = p, b = q;
        while (a != b) {
            a = a == null ? q : a.parent;
            b = b == null ? p : b.parent;
        }
        return a;
    }
}
```

**Time** O(h) | **Space** O(1)